Politechnika Krakowska
Wydział Informatyki i Matematyki
Informatyka – Cyberbezpieczeństwo
II stopień, I rok, II semestr
Przedmiot: *Programowanie równoległe i rozproszone*
Rok akademicki 2026/2027

Wyszukiwanie sygnatur w strumieniu

Aho-Corasick na CPU w OpenMP i MPI oraz PFAC na GPU w CUDA

Projekt C02

Emilia Trybała Piotr Waluszek

5 listopada 2026

Streszczenie

Projekt dotyczy wyszukiwania wielu sygnatur, czyli stałych ciągów bajtów, w dużym strumieniu danych. To podstawowa operacja systemów wykrywania włamań. Przygotowaliśmy referencję sekwencyjną opartą na automacie Aho-Corasick w układzie gęstym i skompresowanym, wersję OpenMP z podziałem tekstu na fragmenty z zakładką $L_{\max} - 1$ i regułą właściciela pozycji startowej, wersję MPI z rozsyłaniem tekstu i wymianą zakładek oraz wersję CUDA opartą na algorytmie PFAC. Wszystkie wersje dają identyczny zbiór trafień. Sprawdzają to testy na danych z własnego generatora, który zna pozycje trafień, oraz suma kontrolna zgodna w ponad 4000 pomiarach na dwóch maszynach. Na M1 Pro z ośmioma rdzeniami wydajnymi i dwoma energooszczędnymi OpenMP osiąga przyspieszenie 8,5, a MPI uruchomione lokalnie 8,2. Przeplot ośmiu niezależnych ścieżek automatu w jednym wątku podnosi przepustowość rdzenia od 5,3 do 6 razy, a wersji OpenMP do 10,5 GB/s, czyli 45 razy powyżej referencji. Kernel PFAC na karcie Tesla T4 osiąga 27 GB/s, 162 razy więcej niż referencja na tej samej maszynie, ale po doliczeniu transferu przez PCIe tylko od 6 do 9 GB/s. W dodatku dla pary pokazujemy krzywą czasu w kształcie litery U w funkcji rozmiaru kawałka. Przy nierównym koszcie danych małe kawałki przydzielane dynamicznie są do 1,14 raza szybsze od podziału na równe bloki. Odpowiadając na pytanie badawcze, pokazujemy, że długie wzorce i częste częściowe dopasowania zmniejszają przewagę obliczeniową GPU z 2,6–2,7-krotnej do 1,2-krotnej, ale o wyborze urządzenia decyduje przede wszystkim koszt przesłania danych.

Spis treści

1 Wstęp **3**

2 Algorytmy **4**
2.1 Automat Aho-Corasick 4
2.2 PFAC, czyli Aho-Corasick bez porażek na GPU 4
2.3 Podział tekstu: zakładka i właściciel pozycji 4
2.4 Tryb zliczania i suma kontrolna 5

3 Dane: generator o kontrolowanych parametrach **5**

4 Implementacje **6**
4.1 Referencja sekwencyjna 7
4.2 OpenMP 7

4.3 Przeplot ścieżek automatu w jednym wątku . . . 7
4.4 MPI . . . 8
4.5 CUDA: PFAC . . . 8

5 Poprawność **9**

6 Środowisko i metodyka pomiarów **10**
6.1 Metodyka . . . 10

7 Wyniki **12**
7.1 Przepustowość a rozmiar tekstu . . . 12
7.2 Liczba wzorców i układ automatu . . . 13
7.3 Długość wzorców i częstość trafień . . . 15
7.4 OpenMP: przyspieszenie i efektywność . . . 17
7.5 MPI lokalne . . . 18
7.6 Przeplot ścieżek: ile da się wycisnąć z jednego rdzenia . . . 19
7.7 GPU: PFAC na karcie Tesla T4 . . . 21

8 Dodatek dla pary: podział tekstu i krzywa U **24**

9 Pytanie badawcze: CPU czy GPU? **26**

10 Podsumowanie i wnioski **28**
10.1 Ograniczenia . . . 29
10.2 Podział pracy . . . 30

A Odtwarzalność **30**

B Interfejs programów **31**

Bibliografia **32**

1 Wstęp

Systemy wykrywania i zapobiegania włamaniom, takie jak Snort [1] i Suricata [2], a także skanery plików i zapisanego ruchu sieciowego sprawdzają, czy w strumieniu bajtów występuje któryś z tysięcy znanych ciągów, czyli *sygnatur*. Rdzeniem takiego silnika jest wyszukiwanie wielu wzorców jednocześnie. Mamy zbiór literalnych wzorców bajtowych $P = \{p_0, \ldots, p_{k-1}\}$ i tekst T o długości n, a szukamy wszystkich par złożonych z numeru wzorca i pozycji, od której ten wzorzec występuje w tekście. Od szybkości tego kroku zależy, ile ruchu jeden węzeł jest w stanie przeanalizować. Dlatego celem projektu jest jak największa przepustowość skanowania przy wyniku identycznym z implementacją referencyjną, również dla wzorców nakładających się na siebie i przecinających granice fragmentów tekstu.

Zakres. Zgodnie z kartą tematu C02 przygotowaliśmy:

- **referencję sekwencyjną**: automat Aho-Corasick [3] w dwóch układach pamięci, jako gęstą tablicę przejść DFA i jako automat skompresowany;
- **wersję OpenMP** [4]: podział tekstu na bloki z zakładką $L_{\max} - 1$ i regułą właściciela pozycji startowej, w czterech wariantach przydziału pracy;
- **wersję MPI** [5]: rozesłanie wzorców i rozłącznych części tekstu, wymianę zakładek między procesami i scalenie wyników z pozycjami globalnymi, z osobnym pomiarem każdej fazy;
- **wersję CUDA** [6]: algorytm PFAC [7], w którym każdy wątek zaczyna dopasowanie od własnej pozycji, w pięciu wariantach dostępu do pamięci i z trybem potokowym, w którym transfer danych nakłada się na obliczenia;
- **dodatek dla pary**: porównanie podziału tekstu na kilka dużych bloków przydzielanych statycznie z podziałem na wiele małych kawałków przydzielanych dynamicznie, razem z pełną krzywą czasu w funkcji rozmiaru kawałka (Sekcja 8). Porównaliśmy też drugi układ automatu, który karta podaje jako alternatywę;
- **generator danych** o kontrolowanych parametrach, który zapisuje pozycje wszystkich wstawionych trafień.

Rozszerzenia. Poza wymaganym zakresem zbadaliśmy przeplot ścieżek automatu (ang. *interleaving*). Jeden wątek prowadzi wtedy kilka niezależnych przejść przez automat jednocześnie, co na procesorze M1 Pro daje do sześciu razy wyższą przepustowość na rdzeń (Sekcja 7.6). Pokazuje to, że Aho-Corasick na CPU jest ograniczony opóźnieniem pamięci, a nie mocą obliczeniową. Podajemy też wstępną odpowiedź na pytanie badawcze z karty, czyli czy długość wzorców i udział częściowych dopasowań pozwalają przewidzieć, czy szybszy będzie CPU, czy GPU (Sekcja 9).

Układ raportu. W sekcji 2 opisujemy algorytmy i regułę podziału tekstu, w sekcji 3 generator danych, w sekcji 4 implementacje i decyzje projektowe, a w sekcji 5 sposób sprawdzania poprawności. Sekcja 6 przedstawia sprzęt i metodykę pomiarów, Sekcje 7–9 zawierają wyniki, a Sekcja 10 wnioski i podział pracy. Instrukcja odtworzenia wyników znajduje się w dodatku A. Kod, surowe wyniki i skrypty tworzące wykresy są w repozytorium `https://github.com/PiotrWaluszek/prir-c02-wyszukiwanie-sygnatur`.

2 Algorytmy

2.1 Automat Aho-Corasick

Automat Aho-Corasick [3] powstaje z drzewa trie zbudowanego ze wszystkich wzorców. Stan odpowiada prefiksowi któregoś wzorca, a krawędź dopisaniu jednego bajtu. Dla każdego stanu s wyznacza się *link porażki* $f(s)$, czyli stan odpowiadający najdłuższemu właściwemu sufiksowi napisu s, który jest jednocześnie prefiksem jakiegoś wzorca. Wyznacza się też *link wyjściowy*, który wskazuje najbliższy stan na łańcuchu porażek, w którym kończy się wzorzec. Linki liczymy przeszukiwaniem wszerz od korzenia i w tej samej kolejności numerujemy stany. Dzięki temu płytkie stany, odwiedzane najczęściej, leżą w pamięci obok siebie.

Wyszukiwanie czyta tekst raz, bajt po bajcie, i po każdym bajcie zgłasza wszystkie wzorce kończące się w bieżącym stanie, przechodząc łańcuch linków wyjściowych. Złożoność wynosi $O(n + m + z)$, gdzie $m = \sum |p_j|$ jest sumą długości wzorców, a z liczbą trafień. Czas skanu nie zależy więc od liczby wzorców, tylko od tego, jak szybko procesor wykonuje pojedyncze przejście. W praktyce liczba wzorców wpływa jednak na to, czy automat mieści się w pamięci podręcznej, a przez to na czas jednego przejścia (Sekcja 7.2).

Układ gęsty DFA. Linki porażki wbudowujemy w tablicę przejść, tak że `next[s][c]` od razu wskazuje stan docelowy. Przejście to wtedy jeden odczyt liczby `uint32` z tablicy o rozmiarze $S \times 256$. Najstarszy bit wpisu oznacza stan z wyjściem, więc sprawdzenie trafienia w pętli skanu nie wymaga dodatkowego odczytu. Kosztem jest 1 KiB pamięci na stan. Dla 1000 wzorców o długości od 8 do 32 bajtów automat ma 18 222 stany i razem z danymi o wyjściach zajmuje 18,5 MiB, czyli więcej niż pamięć L2 jednego klastra rdzeni M1 Pro, która ma 12 MiB.

Układ skompresowany. Każdy stan przechowuje 256-bitową mapę obecnych dzieci zapisaną w czterech słowach `u64`, przesunięcie do tablicy dzieci i link porażki, razem 40 B. Indeks dziecka wyznacza instrukcja `popcount` na prefiksie mapy, a gdy dziecka nie ma, przechodzimy po linkach porażki. Pełny wiersz 256 przejść ma tylko korzeń. Taka struktura jest około dwunastu razy mniejsza od gęstej, ale przejście jest nieregularne: zawiera pętlę po linkach porażki i rozgałęzienia zależne od danych.

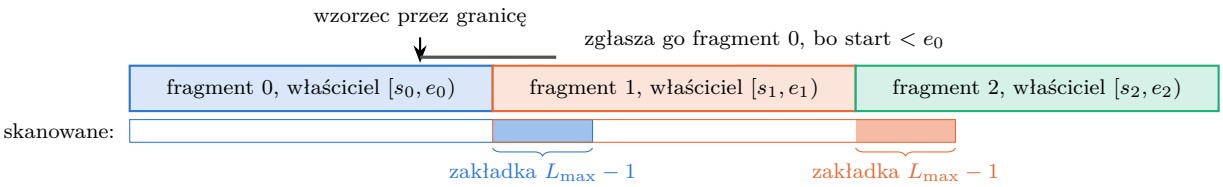
2.2 PFAC, czyli Aho-Corasick bez porażek na GPU

Automat Aho-Corasick jest z natury sekwencyjny, bo stan po bajcie i zależy od stanu po bajcie $i - 1$. Algorytm PFAC [7] odwraca to podejście. **Każdy wątek GPU zaczyna w korzeniu na własnej pozycji startowej** i i idzie wyłącznie krawędziami drzewa trie, aż trafi na brakujące przejście. Linki porażki nie są potrzebne, bo przesunięcie okna załatwia sąsiedni wątek. Wątek i zgłasza tylko wzorce zaczynające się na pozycji i, więc trafienia z definicji się nie powtarzają. Wadą jest nierówna praca wątków. Większość kończy po zerze lub jednym kroku, ale wątki, które trafią na częściowe dopasowanie, idą głębiej, a cały warp czeka na najdłuższy z nich. To zjawisko nazywa się rozbieżnością wątków (*divergence*). Koszt pracy zależy więc od danych, co badamy w sekcjach 7.7 i 9. Inne sposoby przeniesienia Aho-Corasick na GPU w systemach wykrywania włamań opisuje m.in. [8].

2.3 Podział tekstu: zakładka i właściciel pozycji

Wszystkie wersje równoległe dzielą tekst na fragmenty skanowane niezależnie. Przyjmujemy jedną regułę, pokazaną na rysunku 1:

> **Fragment** $[s, e)$ **jest właścicielem pozycji startowych od** s **do** $e - 1$**.** Skanuje bajty $[s, \min(e + L_{\max} - 1, n))$, zaczynając w korzeniu, i zgłasza tylko te trafienia,



Rysunek 1: Reguła właściciela pozycji startowej. Fragment skanuje swoją część i zakładkę długości $L_{\max} - 1$ z następnego fragmentu, ale zgłasza tylko trafienia zaczynające się w swojej części. Tę samą regułę stosujemy w OpenMP, w MPI, w trybie potokowym CUDA i przy przeplocie ścieżek w jednym wątku.

które zaczynają się przed e.

Każde wystąpienie ma dokładnie jedną pozycję startową, więc należy do dokładnie jednego fragmentu i nie powstają ani duplikaty, ani zgubione trafienia. Zakładka długości $L_{\max} - 1$ wystarcza, ponieważ wzorzec zaczynający się najpóźniej na pozycji $e - 1$ kończy się najpóźniej na $e + L_{\max} - 2$. Start w korzeniu bez wcześniejszego rozgrzewania automatu jest poprawny, bo po przeczytaniu $T[s..i]$ automat zna wszystkie wystąpienia zaczynające się od pozycji s lub dalej, a wystąpienia sprzed s należą do poprzedniego fragmentu. Skan zakładki kończymy wcześniej, gdy głębokość bieżącego stanu, czyli długość najdłuższego aktywnego sufiksu, nie sięga już przed e. Od tego momentu żadne dalsze trafienie nie może należeć do tego fragmentu. W praktyce skan zakładki kończy się po jednym lub dwóch bajtach, co widać w zmierzonym narzucie (Sekcja 8).

2.4 Tryb zliczania i suma kontrolna

Przy milionach trafień zapis pełnej listy kosztuje więcej niż samo wyszukiwanie. Karta dopuszcza w takiej sytuacji tryb zliczania, dlatego pomiary wykonujemy w trybie `count`, a tryb `list`, zwracający pełną listę par, służy do testów i do eksperymentu E4. Żeby tryb zliczania nadal wykrywał błędy, oprócz liczby trafień liczymy **sumę kontrolną niezależną od kolejności**:

$$C = \sum_{(j,\,x) \in \text{trafienia}} H(j) \cdot (x + K) \bmod 2^{64}, \qquad H(j) = \text{splitmix64}(j) \mid 1, \tag{1}$$

gdzie K jest stałą. Suma jest przemienna i łączna, więc wątki, procesy i bloki GPU liczą sumy częściowe, a wynik nie zależy od podziału pracy. Zgubione, zdublowane lub przesunięte trafienie zmienia C z prawdopodobieństwem bliskim jedności. Wzór jest liniowy względem x, więc dla każdego stanu automatu można z góry policzyć $\sum H$ i $\sum H \cdot |p|$ po jego wyjściach. Dzięki temu tryb zliczania nie musi przechodzić łańcucha linków wyjściowych.

3 Dane: generator o kontrolowanych parametrach

Zgodnie z kartą danych dostarcza własny generator `sigscan-gen`. Pozwala on sterować cechami, które wpływają na koszt dopasowania, i dla każdego zbioru zapisuje wzorcowy wynik, czyli pozycje wszystkich wstawionych trafień. Wszystkie parametry generatora zestawia Tabela 1.

Jak powstają dane. Tło to losowe bajty z wybranego alfabetu. Długości wzorców są losowane z przedziału od `minlen` do `maxlen`, a część wzorców zaczyna się jednym z kilku wspólnych prefiksów, co wymusza częściowe dopasowania i rozgałęzione ścieżki w automacie. Następnie generator wstawia w losowe, nienachodzące na siebie miejsca pełne wystąpienia wzorców, których pozycje zapisuje w pliku `.truth.csv`, oraz częściowe dopasowania, czyli prefiksy wzorców o losowej długości, po których stoi bajt inny niż kolejny bajt wzorca. Na żądanie wstawia też wystąpienia przecinające każdą granicę bloków. W trybie `--no-match` tło składa się z bajtów mniejszych od `0x80`, a każdy wzorzec zaczyna się bajtem co najmniej `0x80`. Brak trafień jest

Tabela 1: Parametry generatora i ich wartości w konfiguracji bazowej pomiarów. Pełny opis wyświetla polecenie `sigscan-gen --help`.

Parametr	Znaczenie	Bazowo
`--size`	rozmiar tekstu, z przyrostkami k, M, G	512 MiB
`--npat`, `--minlen`, `--maxlen`	liczba wzorców i zakres ich długości, rozkład jednostajny	1000; 8-32 B
`--hit-rate`	liczba wstawionych pełnych trafień na MiB tekstu	10
`--partial-rate`	liczba wstawionych częściowych dopasowań na MiB, czyli prefiksów wzorca o losowej długości, po których dopasowanie się urywa	0
`--prefix-share`, `--prefix-groups`, `--prefix-len`	odsetek wzorców ze wspólnym prefiksem, liczba takich prefiksów i ich długość	0,3; 8; 4
`--alphabet`	alfabet tła i wzorców: `full` z 256 bajtami, `ascii`, `dna` albo liczba symboli	`full`
`--no-match`	dane, w których na pewno nie ma żadnego trafienia	nie
`--boundary-block`	trafienie przecinające każdą granicę bloków o podanym rozmiarze	nie
`--hot-region`	obszar z gęstszymi trafieniami, zapis `początek:długość:mnożnik`	nie
`--seed`	ziarno główne	2026

wtedy gwarantowany, a nie tylko prawdopodobny. Losowe tło może przypadkiem zawierać krótkie wzorce, dlatego test sprawdza, czy wynik referencji *zawiera* wszystkie zapisane trafienia, a nie czy jest im równy (Sekcja 5).

Odtwarzalność. Generator korzysta z trzech niezależnych strumieni losowych: osobnego dla wzorców, dla tła i dla wstawianych trafień. Wszystkie wyprowadza z ziarna funkcją `splitmix64`, więc na przykład zmiana rozmiaru tekstu nie zmienia zbioru wzorców. W trakcie projektu okazało się, że to samo ziarno dawało różne dane na macOS i na Linuksie. Standard C++ nie określa dokładnie, jak działa `std::uniform_int_distribution`, ani w jakiej kolejności `std::sort` ustawia elementy równe, a biblioteki libc++ i libstdc++ robią to inaczej. Zastąpiliśmy je własnym losowaniem metodą Lemire'a [9] i sortowaniem stabilnym. Teraz Mac i Colab tworzą bajtowo identyczne pliki, co sprawdziliśmy sumą SHA-256, więc wyniki obu maszyn dotyczą dokładnie tych samych danych.

Formaty plików. Tekst zapisujemy jako surowe bajty w pliku `.bin`. Plik `.pat` zawiera jeden wzorzec w zapisie szesnastkowym na linię, a numer linii jest identyfikatorem wzorca; znak `#` rozpoczyna komentarz. Trafienia zapisujemy w pliku CSV z kolumnami `pattern_id` i `offset`, posortowane po pozycji. Plik `.meta.json` przechowuje parametry i ziarno. Wszystkie programy czytają te same pliki.

4 Implementacje

Wszystkie wersje napisaliśmy w C++20 i budujemy za pomocą CMake [10]. Wspólna biblioteka `sigscan_core` zawiera parser wzorców, budowę automatów, skan z regułą właściciela, sumę kontrolną, obsługę plików i zapis wyników. Programy `sigscan-seq`, `sigscan-omp`, `sigscan-mpi` i `sigscan-cuda` przyjmują te same podstawowe opcje, opisane w dodatku B. Dla każdego powtórzenia wypisują jeden rekord CSV, w którym czas jest rozbity na odczyt pliku, budowę automatu, transfery, komunikację, skan i scalanie wyników. Cały rdzeń obliczeń to nasz własny kod. Z bibliotek zewnętrznych korzystamy tylko w testach (doctest [11]) i przy rysowaniu wykresów (NumPy [12], pandas [13], Matplotlib [14]).

4.1 Referencja sekwencyjna

Referencja buduje automat w sposób opisany w sekcji 2 i skanuje cały tekst jednym wywołaniem funkcji `scan_chunk`, tej samej, której używają OpenMP i MPI. Pętla główna z listingu 1 wykonuje na każdy bajt jeden odczyt tablicy i jedno porównanie. Obsługę trafień przekazujemy jako parametr szablonu. `CountSink` dodaje wcześniej policzone sumy stanu (Sekcja 2.4), a `ListSink` przechodzi łańcuch linków wyjściowych i dopisuje pary do lokalnego wektora. Czas budowy automatu mierzymy osobno.

Listing 1: Pętla skanu dla pozycji należących do fragmentu, wspólna dla wszystkich wersji CPU.

```
u32 st = 0;                                          // korzeń
for (u64 i = 0; i < own_end; ++i) {
    st = ac.step(st & kMask, buf[i]);                // DFA: next[st * 256 + bajt]
    if (st & kFlag) [[unlikely]]                     // stan z wyjściem
        sink.on_state(ac.out, st & kMask, base + i);
}
return finish_overlap(ac, buf, buf_len, own_end, base, sink, st);   // zakładka
```

4.2 OpenMP

Tekst dzielimy na kawałki $[s_k, e_k)$ według reguły właściciela, a kawałek k skanuje bajty $[s_k, \min(e_k + L_{\max} - 1, n))$. Porównujemy cztery sposoby przydziału pracy:

- **bloki**: p równych bloków, po jednym na wątek, czyli kilka dużych bloków przydzielanych statycznie, o których mówi karta;
- **static**, **dynamic** i **guided**: kawałki o stałym rozmiarze C, ustawianym opcją `--chunk`, rozdzielane przez `#pragma omp for schedule(runtime)`. Harmonogram wybieramy funkcją `omp_set_schedule`. To odpowiada wielu małym kawałkom przydzielanym dynamicznie.

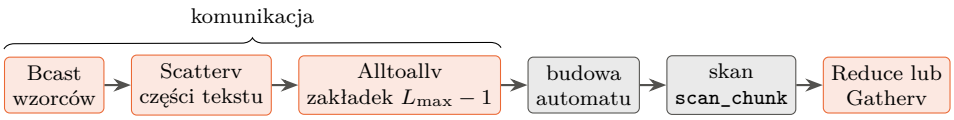
Każdy wątek zbiera wyniki we własnej strukturze wyrównanej do 64 B, dzięki czemu wątki nie współdzielą linii pamięci podręcznej. W pętli skanu **nie ma żadnej synchronizacji**: nie ma sekcji krytycznych ani operacji atomowych, a jedyną barierą jest koniec regionu równoległego. Po nim wątek główny sumuje liczniki i sumy kontrolne albo łączy listy, a czas tego scalania raportujemy osobno. Program zlicza też bajty skanowane ponownie w zakładkach, co wykorzystujemy w analizie krzywej U (Sekcja 8). Na macOS nie da się przypiąć wątków do rdzeni, bo zmienna `OMP_PROC_BIND` nie działa. O tym, który wątek trafi na rdzeń wydajny, a który na energooszczędny, decyduje więc system.

4.3 Przeplot ścieżek automatu w jednym wątku

W pętli z listingu 1 adres kolejnego odczytu tablicy zależy od wyniku poprzedniego. Gdy tablica nie mieści się w pamięci podręcznej L1 ani L2, każdy bajt czeka na odpowiedź pamięci, a jednostki wykonawcze procesora stoją bezczynnie. Funkcja `scan_interleaved<K>` dzieli kawałek na K równych pasów i przesuwa wszystkie K stanów jednocześnie. Każdy pas jest osobnym fragmentem w rozumieniu reguły właściciela.

```
for (u64 i = 0; i < L; ++i)
    for (int j = 0; j < K; ++j) {                    // K niezależnych odczytów naraz
        st[j] = ac.step(st[j] & kMask, buf[j * L + i]);
        if (st[j] & kFlag) [[unlikely]] sink.on_state(ac.out, st[j] & kMask, base +
    j * L + i);
    }
```

Odczyty z różnych pasów nie zależą od siebie, więc procesor może obsługiwać je równolegle.



Rysunek 2: Fazy jednego powtórzenia w wersji MPI. Każda faza jest otoczona barierą, a jej czas mierzymy funkcją `MPI_Wtime`. Fazy komunikacji są pomarańczowe, a obliczenia lokalne szare.

Wynik jest identyczny ze zwykłym skanem, co sprawdzają testy z sekcji 5. Opcja `--interleave` działa w wersji sekwencyjnej i w OpenMP.

4.4 MPI

Pliki wczytuje proces o numerze 0. Każde powtórzenie pomiaru składa się z sześciu faz rozdzielonych barierami, pokazanych na rysunku 2. Czas fazy to czas najwolniejszego procesu.

1. **Wzorce.** `MPI_Bcast` rozsyła rozmiar i zserializowany zbiór wzorców.
2. **Tekst.** `MPI_Scatterv` rozsyła rozłączne części $[s_r, e_r)$, więc żaden bajt nie jest wysyłany dwa razy.
3. **Zakładki.** Proces r potrzebuje bajtów $[e_r, e_r + L_{\max} - 1)$, które należą do kolejnych procesów. Wymieniamy je przez `MPI_Alltoallv`, bo gdy części są krótsze niż $L_{\max}$, zakładka obejmuje kilku sąsiadów. MPI nie pozwala, by bufor nadawczy i odbiorczy nachodziły na siebie, dlatego zakładka trafia najpierw do osobnego bufora, a potem jest dopisywana.
4. **Budowa automatu.** Każdy proces równolegle buduje własną kopię.
5. **Skan.** Funkcja `scan_chunk` dostaje przesunięcie globalne s_r.
6. **Wyniki.** `MPI_Reduce` sumuje liczbę trafień i sumę kontrolną, bo dodawanie modulo 2^{64} jest łączne. W trybie listy `MPI_Gatherv` zbiera pary z pozycjami globalnymi.

Czas komunikacji to suma trzech pierwszych faz, a czas scalania to faza szósta. Wszystkie pomiary MPI wykonaliśmy na jednym komputerze, gdzie procesy komunikują się przez pamięć współdzieloną. Są to więc **wyniki lokalne**, które nie mówią nic o skalowaniu na klastrze.

4.5 CUDA: PFAC

Na kartę graficzną kopiujemy tablicę PFAC i dane o wyjściach stanów. Tablica PFAC to gęsta tablica $S \times 256$ zawierająca wyłącznie krawędzie drzewa trie. Wartość 0 oznacza brak krawędzi, bo żadna krawędź nie prowadzi do korzenia. Dane o wyjściach to liczba wzorców kończących się w danym stanie i suma ich wartości H dla trybu zliczania, a w trybie listy także identyfikatory wzorców. Wątek i przechodzi ścieżkę PFAC od pozycji i, co zajmuje najwyżej $L_{\max}$ kroków, i liczy trafienia lokalnie. Wyniki sumujemy najpierw w obrębie warpu instrukcją `__shfl_down_sync`, potem w bloku przez pamięć współdzieloną, a na końcu jeden wątek bloku wykonuje `atomicAdd`. Liczba bloków jest stała i jest wielokrotnością liczby multiprocesorów, a bloki przechodzą przez tekst pętlą typu *grid-stride*. Porównujemy pięć wariantów kernela:

- `global`: tekst i tablica czytane zwykłymi odczytami z pamięci globalnej;
- `ldg`: tablica czytana przez pamięć podręczną tylko do odczytu, instrukcją `__ldg`;
- `smem-text`: blok kopiuje do pamięci współdzielonej kafel tekstu o długości blockDim + $L_{\max} - 1$ bajtów, a wątki czytają znaki z kafla;

- `smem-root`: w pamięci współdzielonej leży wiersz korzenia, czyli 256 wpisów potrzebnych w pierwszym kroku każdego wątku;
- `smem-all`: połączenie trzech poprzednich.

Tryb listy działa w dwóch przebiegach. Pierwszy zlicza trafienia, potem rezerwujemy bufor dokładnie potrzebnej wielkości, a drugi przebieg zapisuje do niego pary, korzystając z atomowego licznika. Dzięki temu bufor nigdy się nie przepełnia i nie trzeba zgadywać jego rozmiaru. **Tryb potokowy**, włączany opcją `--streams`, dzieli tekst na kawałki z zakładką. Dla kawałka c zlecamy kopiowanie z pamięci przypiętej (ang. *pinned*) funkcją `cudaMemcpyAsync` i uruchomienie kernela w strumieniu numer $c \bmod k$. Kopiowanie kolejnych kawałków odbywa się więc w tym samym czasie co obliczenia na poprzednich. Czas mierzymy zdarzeniami `cudaEvent` po synchronizacji urządzenia, osobno dla kopiowania na kartę tekstu i automatu, dla kernela i dla kopiowania wyników z powrotem. Pamięć na tekst i automat rezerwujemy raz, poza pomiarem. Wyjątkiem jest bufor na pary w trybie listy, którego rozmiar znamy dopiero po pierwszym przebiegu. Rezerwujemy go w każdym powtórzeniu, a ten koszt razem z drugim przebiegiem kernela wliczamy do czasu kopiowania wyników. Opcja `--verify` porównuje wynik GPU z referencją na CPU i przy różnicy kończy program błędem.

5 Poprawność

Poprawność sprawdzamy na trzech poziomach. Żaden pomiar nie zaczyna się, jeśli testy nie przechodzą, a skrypt sesji GPU przerywa pracę przy pierwszym błędzie.

1. **Wynik naiwny jest równy wynikowi Aho-Corasick** na małych danych. Algorytm naiwny o złożoności $O(n \cdot \sum |p_j|)$ porównuje każdy wzorzec na każdej pozycji i służy jako wyrocznia dla wszystkich układów automatu.
2. **Wynik referencji zawiera wszystkie trafienia wstawione przez generator.** Może zawierać więcej, bo w losowym tle zdarzają się przypadkowe wystąpienia.
3. **Każda wersja równoległa daje to samo co referencja.** W trybie listy porównujemy zbiory par złożonych z numeru wzorca i pozycji, a w trybie zliczania liczbę trafień i sumę kontrolną (1).

Testy jednostkowe. Mamy 20 przypadków testowych z 16 478 asercjami, napisanych w bibliotece doctest. Każdy przypadek przechodzi przez wszystkie układy automatu, czyli gęsty, skompresowany i emulację PFAC na CPU, przez oba tryby wyniku, przez podział na kawałki o rozmiarach 1, 2, 3, 5, 7, 8, 13 i 64 bajty, w tym mniejsze niż $L_{\max}$, oraz przez przeplot z K równym 2, 4, 8 i 16. Sprawdzamy następujące przypadki brzegowe:

- klasyczny zestaw wzorców `he`, `she`, `his`, `hers`;
- nakładające się wystąpienia, na przykład `aa` w `aaaa` daje trzy trafienia;
- wzorzec, który jest prefiksem, sufiksem lub środkiem innego wzorca;
- powtórzone wzorce, które dostają osobne identyfikatory;
- bajty `0x00`, `0xFF` i znak nowej linii;
- wzorzec jednobajtowy, wzorzec dłuższy niż tekst, pusty tekst i wzorzec dłuższy niż kawałek.

Do tego dochodzi 200 losowych przypadków z alfabetem od dwóch do czterech symboli, w których wzorce często się nakładają. Testy generatora sprawdzają, że to samo ziarno daje te same dane,

że zbiór wzorców nie zależy od rozmiaru tekstu, że tryb `--no-match` rzeczywiście nie zawiera trafień i że każde trafienie na granicy bloków zostaje znalezione dokładnie raz.

Test integracyjny. Skrypt `tests/integration.py` tworzy sześć scenariuszy z Tabela 2 i na każdym uruchamia wszystkie zbudowane warianty programów, porównując ich wynik z referencją.

Tabela 2: Scenariusze testu integracyjnego.

Scenariusz	Co sprawdza
`basic`	2 MiB, 300 wzorców o długości 4-24 B, trafienia, częściowe dopasowania i wspólne prefiksy
`boundary`	trafienie przecinające każdą granicę co 4 KiB, wzorce do 64 B
`nomatch`	dane bez żadnego trafienia
`dense-hits`	alfabet z trzech symboli i wzorce o długości 1-6 B, co daje 633 458 trafień na 512 KiB
`long-patterns`	wzorce o długości 100-600 B, dłuższe niż kawałki i części procesów MPI
`dna`	alfabet A, C, G, T, 1000 wzorców, 80% z nich ze wspólnym prefiksem

Sprawdzane warianty to referencja w obu układach automatu, emulacja PFAC i przeplot z K równym 2, 4, 8 i 16. Dla OpenMP sprawdzamy podział na bloki przy 1, 2, 3, 7 i 10 wątkach oraz harmonogramy `static`, `dynamic` i `guided` z kawałkami 3 B, 1 KiB i 64 KiB. MPI uruchamiamy z 1, 2, 3, 4 i 7 procesami, także wtedy, gdy części są krótsze niż $L_{\max}$. Na Colabie dochodzi pięć kerneli CUDA i tryb potokowy z kawałkami 1 KiB i 100 KiB. Łącznie jest to 31 wariantów na Macu i 38 na Colabie. Każdy sprawdzamy w obu trybach wyniku, poza trybem potokowym, który działa tylko w trybie zliczania.

Wynik. Wszystkie testy przechodzą na obu maszynach, czyli na macOS z procesorem ARM i na Linuksie z procesorem x86-64 i kartą T4. Każde z porównań w sześciu scenariuszach dla 38 wariantów daje dokładnie ten sam wynik. Dodatkowo każdy pomiar zapisuje liczbę trafień i sumę kontrolną, a skrypt `check_results.py` sprawdza, że na tym samym zbiorze danych wszystkie wersje dają te same wartości. Przy pierwszym uruchomieniu na GPU wszystkie warianty kernela zgodziły się z referencją bez żadnych poprawek.

6 Środowisko i metodyka pomiarów

Sprzęt i oprogramowanie obu maszyn zestawia Tabela 3. OpenMP i MPI mierzymy wyłącznie na MacBooku, bo Colab ma tylko dwa wirtualne rdzenie. Na Colabie uruchamiamy natomiast wersję sekwencyjną i OpenMP z dwoma wątkami jako referencję CPU na tej samej maszynie co GPU. Dzięki temu przyspieszenie GPU można podać względem jawnie wskazanej referencji. Kod CUDA kompilujemy dla architektury `sm_75`. Sesją Colab sterujemy z terminala za pomocą oficjalnego narzędzia Colab CLI [18]. Skrypt `bench/colab/run_gpu.sh` tworzy maszynę z kartą T4, wysyła na nią kod z repozytorium, buduje go, uruchamia testy i po każdej serii pobiera plik z wynikami. Jeśli sesja zostanie przerwana, tracimy najwyżej jedną serię.

6.1 Metodyka

- **Dane** generujemy przed serią, poza pomiarem, i zapisujemy pod skrótem parametrów. Wszystkie wersje i obie maszyny pracują więc na tych samych plikach.
- **Powtórzenia.** Każde uruchomienie programu zawiera jeden przebieg rozgrzewkowy i **siedem mierzonych**. Na wykresach punkt oznacza **medianę**, a słupki rozstęp między-

Tabela 3: Sprzęt i oprogramowanie. Pełny opis każdej maszyny zapisuje skrypt `bench/env_info.sh` w katalogu `results/env`.

	MacBook Pro: OpenMP, MPI, CPU	Google Colab: CUDA
Procesor	Apple M1 Pro, 8 rdzeni wydajnych P i 2 energooszczędne E, bez SMT	Intel Xeon 2,0 GHz Skylake-SP, 2 wirtualne rdzenie, czyli 1 rdzeń z SMT
Pamięć podręczna	L1d 128 KiB na rdzeń P, L2 12 MiB na klaster P	L1d 32 KiB, L2 1 MiB, L3 38,5 MiB
Pamięć	16 GiB LPDDR5, pamięć wspólna dla CPU i GPU	12 GiB
GPU	nie używane	NVIDIA Tesla T4, architektura Turing `sm_75`, 40 multiprocesorów, 15 GiB GDDR6, L2 4 MiB, PCIe 3.0 x16
System	macOS 27.0	Ubuntu 24.04, sterownik 580.82, CUDA 13.0
Kompilator	Apple clang 16 i libomp 20.1 [15], `-O3 -mcpu=native`	g++ 13.3 `-O3 -march=native`, nvcc 13.0 [16]
MPI	Open MPI 5.0.11 [17] na jednej maszynie	nie używane
Zasilanie	zasilacz podłączony, tryb oszczędzania energii wyłączony, ekran wygaszony	nie dotyczy

kwartylowy, czyli przedział od pierwszego do trzeciego kwartyla. Surowe pliki CSV zawierają wszystkie powtórzenia.

- **Kolejność.** Przed serią generujemy wszystkie jej zbiory danych, a potem losujemy ze stałym ziarnem kolejność wszystkich uruchomień, także między zbiorami. Dzięki temu powolne zmiany stanu maszyny, na przykład nagrzewanie się procesora, nie zniekształcają zależności od żadnego badanego parametru. Między uruchomieniami robimy dwusekundową przerwę. Serie GPU losowały kolejność tylko w obrębie zbioru danych, ale na Colabie te same konfiguracje kernela w różnych seriach różnią się o najwyżej 1%.

- **Warunki pomiarów CPU.** Pierwsza seria pomiarów na Macu szła wtedy, gdy na komputerze działały równolegle inne programy, w tym skrypty tworzące wykresy. Przy 8-10 wątkach te same konfiguracje zmierzone w różnych seriach różniły się nawet o 25%. Wszystkie serie CPU powtórzyliśmy więc przy podłączonym zasilaczu, z wygaszonym ekranem i wstrzymanymi na ten czas procesami macOS działającymi w tle, takimi jak indeksowanie Spotlight i analiza zdjęć. Seria startowała dopiero wtedy, gdy obciążenie systemu przez kilkanaście sekund nie przekraczało 30% jednego rdzenia. Początek serii E5 trafił mimo to na chwilowe spowolnienie, widoczne także w czasie budowy automatu, więc tę serię powtórzyliśmy w całości. Teraz te same konfiguracje zmierzone w różnych seriach różnią się o najwyżej 3%. Pierwotne wyniki zostawiliśmy w katalogu `results/raw/archive-cpu-obciazony-mac`, ale nie używamy ich w raporcie.

- **Ziarno danych.** Algorytm jest deterministyczny, ale dane są losowe. W dodatkowej serii `e12_seeds` zmierzyliśmy referencję, OpenMP z przeplotem i bez niego oraz MPI na danych bazowych z ziarnami 2026, 2027 i 2028, czyli z innymi wzorcami i innym tekstem. Mediany różnią się o najwyżej 3%, więc pozostałe serie wykonujemy z jednym ziarnem.

- **Zakres mierzonego czasu.** Osobno podajemy **czas skanu**, czyli sam skan na CPU albo sam kernel na GPU przy danych i automacie już obecnych na karcie, oraz **czas pełny** (ang. *end-to-end*). Czas pełny liczymy od danych w pamięci RAM do wyniku na hoście i obejmuje on budowę automatu, komunikację MPI albo kopiowanie danych na kartę i z powrotem, skan oraz scalanie wyników. Odczyt pliku z dysku mierzymy, ale nie wliczamy go do porównań, bo zależy od pamięci podręcznej systemu plików, a nie od algorytmu.

- **Przepustowość** to liczba bajtów tekstu podzielona przez czas, podawana w GB/s, gdzie 1 GB to 10^9 bajtów. Rozmiary danych podajemy w jednostkach binarnych, gdzie 1 MiB to 2^{20} bajtów.
- **Przyspieszenie** $S(p) = T(1)/T(p)$ i efektywność $E(p) = S(p)/p$ liczymy z median przy stałym problemie: 512 MiB tekstu i 1000 wzorców. $T(1)$ to czas referencji sekwencyjnej na tej samej maszynie, a nie wersji równoległej uruchomionej z jednym wątkiem.
- **Synchronizacja.** Na GPU czas mierzą zdarzenia CUDA, a odczytujemy go dopiero wtedy, gdy karta skończy pracę. Czekamy na ostatnie zdarzenie pomiaru, a w trybie potokowym na wszystkie strumienie funkcją `cudaDeviceSynchronize`. W MPI każdą fazę otaczają bariery. W OpenMP mierzymy czas całego regionu równoległego zegarem `steady_clock`.
- **Tryb wyniku.** We wszystkich porównywanych wersjach domyślnie używamy trybu `count`. Tryb `list` mierzymy osobno w eksperymencie E4.
- **Konfiguracja bazowa**, w której zmieniamy jeden parametr naraz: 512 MiB tekstu, 1000 wzorców o długości od 8 do 32 bajtów, 10 trafień na MiB, 30% wzorców ze wspólnym prefiksem, pełny alfabet i ziarno 2026.

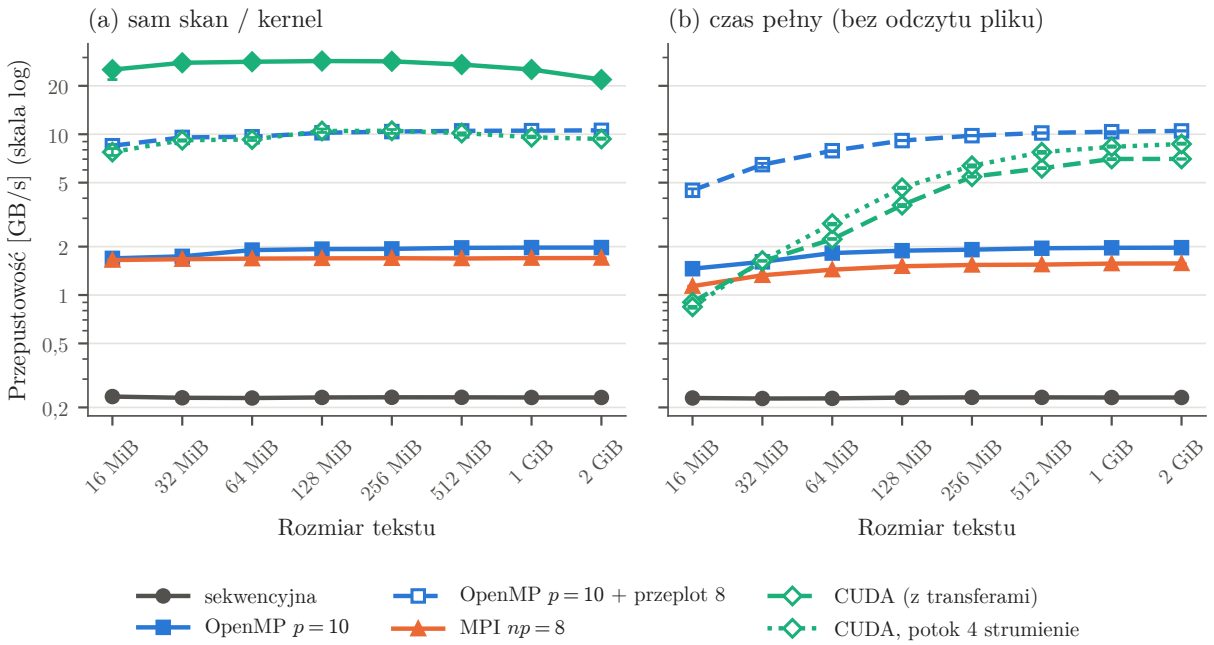
7 Wyniki

Jeśli nie zaznaczono inaczej, punkty na wykresach to mediany z siedmiu powtórzeń, słupki błędów pokazują rozstęp międzykwartylowy, a dane pochodzą z konfiguracji bazowej opisanej w sekcji 6.1. Wszystkie pomiary przeszły kontrolę zgodności. Na każdym zbiorze danych każda wersja, konfiguracja, powtórzenie i maszyna dały tę samą liczbę trafień i tę samą sumę kontrolną, co sprawdza skrypt `scripts/check_results.py`.

7.1 Przepustowość a rozmiar tekstu

Przepustowość skanu wersji CPU prawie nie zależy od rozmiaru tekstu (Rysunek 3a). Referencja osiąga 0,23 GB/s dla wszystkich rozmiarów od 16 MiB do 2 GiB, OpenMP i MPI z ośmioma do dziesięciu wykonawcami od 1,5 do 2,0 GB/s, a OpenMP z przeplotem około 10,5 GB/s. Skan jest liniowy względem długości tekstu, a automat o rozmiarze 18,5 MiB i tak nie mieści się w L2, więc rozmiar tekstu nie zmienia zachowania pamięci podręcznej. Wyjątkiem są najmniejsze teksty w wersjach OpenMP z dziesięcioma wątkami, które dla 16 MiB tracą od 14 do 19%. Tekst dzieli się wtedy tylko na 16 kawałków po 1 MiB, których nie da się równo rozdzielić między dziesięć wątków. Kernel GPU osiąga od 22 do 28,5 GB/s, przy czym najmniej dla 2 GiB, z powodu limitu mocy karty opisanego w sekcji 7.7.

Czas pełny na rysunku 3b pokazuje koszty stałe. Na CPU jest to budowa automatu, trwająca 1,5 ms. Widać ją tylko przy przeplocie, gdzie dla 16 MiB przepustowość spada z 8,5 do 4,5 GB/s. MPI płaci dodatkowo za rozesłanie tekstu. Na GPU koszty stałe są największe. Budowa automatu na CPU Colaba i przesłanie tablicy PFAC o rozmiarze 17,8 MiB sprawiają, że dla 16 MiB GPU osiąga w czasie pełnym tylko 0,9 GB/s, mniej niż zwykłe OpenMP. Przy 32 MiB GPU dorównuje OpenMP bez przeplotu, od 64 MiB jest od niego szybsze, a dla 1 i 2 GiB zbliża się do granicy wyznaczonej przez łącze PCIe: 7,0 GB/s przy jednym transferze i do 8,7 GB/s w trybie potokowym. OpenMP z przeplotem na M1 Pro jest w czasie pełnym szybsze od GPU dla wszystkich rozmiarów.



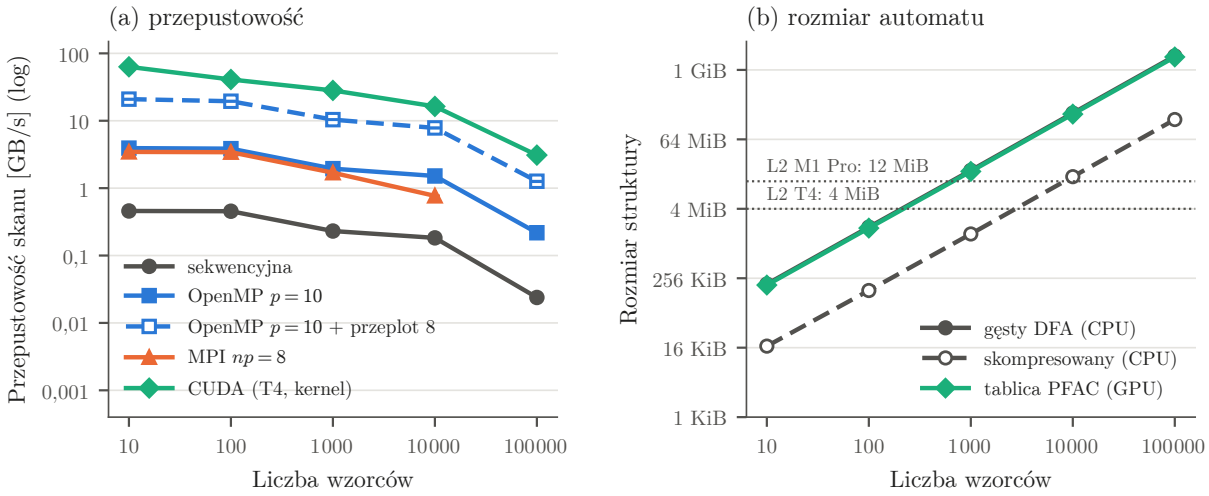
Rysunek 3: Przepustowość w funkcji rozmiaru tekstu dla 1000 wzorców o długości 8-32 B. Wersje CPU działają na M1 Pro, CUDA na karcie Tesla T4 w Colabie, wszystkie na tych samych danych. Panel (a) pokazuje sam skan lub sam kernel, panel (b) czas pełny bez odczytu pliku, czyli z budową automatu, komunikacją MPI albo kopiowaniem danych na kartę, skanem i scalaniem wyników.

7.2 Liczba wzorców i układ automatu

Liczba wzorców wyznacza rozmiar automatu, a ten decyduje, w którym poziomie pamięci odbywa się skan (Rysunek 4). Gęsty DFA zajmuje 1 KiB na stan, co daje 0,2 MiB dla 10 wzorców, 18,5 MiB dla 1000 i 1,7 GiB dla 100 000. Dopóki automat mieści się w L2, czyli do 100 wzorców, referencja osiąga 0,45 GB/s. Dla 1000 wzorców przepustowość spada o połowę, a dla 100 000 do 0,024 GB/s, bo prawie każde przejście chybia w pamięci podręcznej i w buforze translacji adresów. Ten sam kształt mają wszystkie wersje, łącznie z GPU (Sekcja 7.7). Przeplot pomaga w całym zakresie: OpenMP z $K = 8$ jest od 5,1 do 5,8 raza szybsze od zwykłego OpenMP. Wersja MPI dla 10 000 wzorców traci więcej niż OpenMP i osiąga 0,77 GB/s wobec 1,51 GB/s. Każdy z ośmiu procesów ma własną kopię automatu o rozmiarze 182 MiB, więc kopie konkurują o pamięć podręczną ostatniego poziomu, podczas gdy wątki OpenMP korzystają z jednej. Dla 100 000 wzorców osiem kopii po 1,7 GiB zajęłoby prawie 14 GiB z 16 GiB pamięci laptopa, a razem z systemem, tekstem i strukturami potrzebnymi przy budowie przekroczyłoby ją. Dlatego tego punktu dla MPI nie mierzyliśmy.

Czas budowy a czas wyszukiwania. Karta wymaga, żeby oba czasy mierzyć osobno. Zestawia je Tabela 4. Dla zbiorów do 10 000 wzorców budowa na M1 Pro trwa od ułamka milisekundy do kilkudziesięciu milisekund. Dla 10 wzorców jest ponad 20 000 razy krótsza niż skan 256 MiB tekstu, a dla 10 000 wzorców około 70 razy krótsza. Dopiero przy 100 000 wzorców budowa zaczyna się liczyć: gęsty DFA powstaje w 0,5 s, a tablica PFAC na wolniejszym procesorze Colaba w 1,8 s.

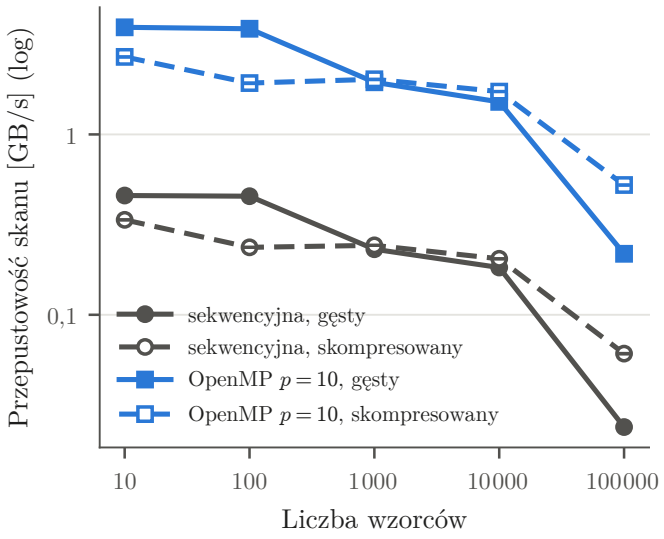
Gęsty czy skompresowany? Automat skompresowany jest około dwunastu razy mniejszy, na przykład dla 1000 wzorców zajmuje 1,5 MiB zamiast 18,5 MiB. Buduje się też dwa razy szybciej, bo dla 100 000 wzorców zajmuje to 240 ms zamiast 494 ms. Jego przejście jest jednak droższe (Rysunek 5). Dla małych zbiorów, od 10 do 100 wzorców, gdy oba układy mieszczą się w L2, szybszy jest układ gęsty: dla 10 wzorców wersja sekwencyjna osiąga 0,46 wobec 0,34 GB/s. Przy



Rysunek 4: Panel (a): przepustowość skanu w funkcji liczby wzorców dla 256 MiB tekstu i wzorców o długości 8-32 B. Panel (b): rozmiar gęstego DFA i automatu skompresowanego używanych na CPU oraz tablicy PFAC używanej na GPU, razem z pojemnością pamięci L2 obu procesorów. DFA i tablica PFAC mają po 1 KiB na stan, więc ich linie się pokrywają.

Tabela 4: Czas budowy automatu i czas skanu 256 MiB tekstu przez referencję sekwencyjną w obu układach. DFA i automat skompresowany budujemy na M1 Pro, tablicę PFAC na procesorze Colaba.

		Budowa automatu [ms]			Skan 256 MiB [ms]	
Wzorce	Stany	DFA	skompr.	PFAC	DFA	skompr.
10	195	0,02	0,02	0,2	586	799
100	1 895	0,15	0,09	1,2	591	1 135
1 000	18 222	1,51	0,78	12,0	1 162	1 106
10 000	179 631	20,61	9,28	144	1 468	1 313
100 000	1 758 205	494,24	240,09	1 832	11 253	4 406



Rysunek 5: Gęsty DFA a automat skompresowany: przepustowość skanu w funkcji liczby wzorców na M1 Pro, dla 256 MiB tekstu i bez przeplotu.

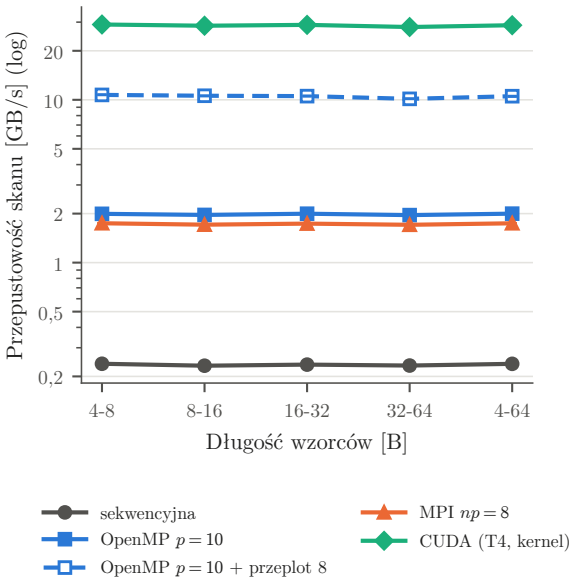
1000 wzorców oba układy są prawie równie szybkie, a przy 100 000 skompresowany jest 2,6 raza szybszy, bo zajmuje 141 MiB zamiast 1,7 GiB. Wybór zależy jednak od tego, czy używamy przeplotu. Z przeplotem $K = 8$ gęsty DFA wygrywa we wszystkich zmierzonych punktach, nawet dla 100 000 wzorców, gdzie OpenMP osiąga 1,27 wobec 0,52 GB/s, ponieważ układ skompresowany z przeplotu nie korzysta (Sekcja 7.6). W praktyce, gdy liczy się szybkość, najlepszy jest DFA z przeplotem, o ile automat mieści się w pamięci. Układ skompresowany warto wybrać, gdy liczy się zajęta pamięć, na przykład przy wielu kopiach automatu w procesach MPI, albo czas budowy.

7.3 Długość wzorców i częstość trafień

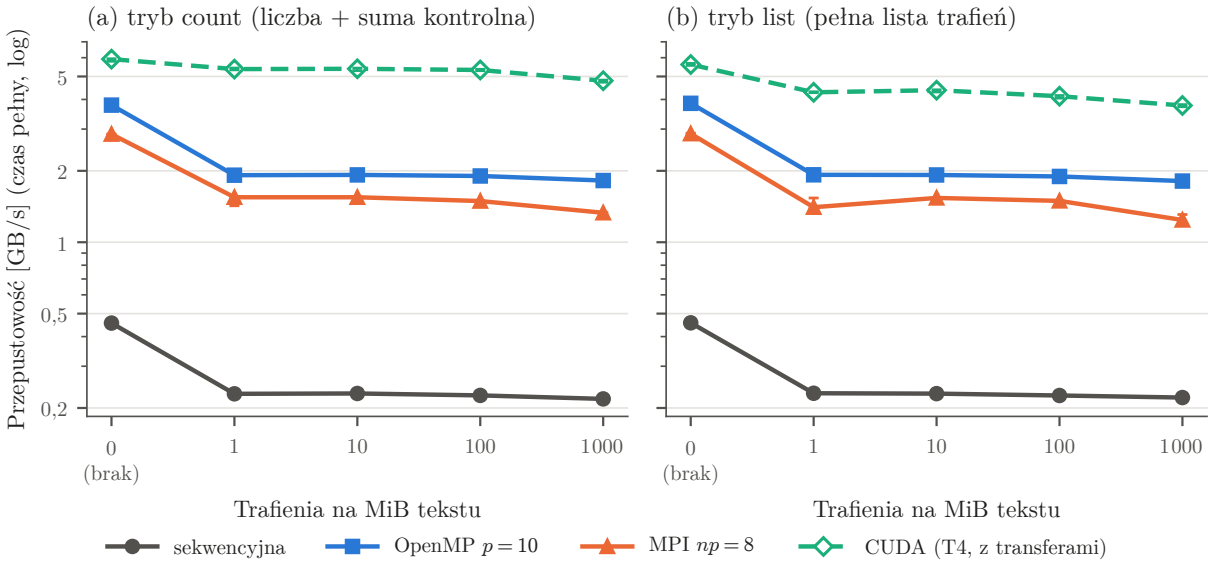
Długość wzorców prawie nie wpływa na przepustowość skanu żadnej wersji (Rysunek 6). Różnice między wzorcami o długości 4-8 B i 32-64 B nie przekraczają 4%, a dla OpenMP z przeplotem 6%. Zgadza się to z teorią. Aho-Corasick wykonuje jedno przejście na bajt niezależnie od długości wzorców, a w PFAC na losowym tle prawie każdy wątek kończy po pierwszym lub drugim kroku, więc dłuższe wzorce nie wydłużają ścieżek. Długość wzorców wpływa jedynie na koszt zakładki, który dzięki wczesnemu zakończeniu jej skanu jest pomijalny (Sekcja 8). W czasie pełnym długość wzorców ma jednak znaczenie, bo wydłuża budowę automatu. Na CPU z przeplotem przepustowość spada z 10,5 do 8,7 GB/s, a na GPU, gdzie dochodzi przesłanie tablicy PFAC o rozmiarze od 4,3 do 45 MiB, z 7,3 do 3,4 GB/s. Długie wzorce spowalniają sam skan dopiero wtedy, gdy tekst często zawiera ich długie prefiksy. Ten przypadek badamy w Sekcja 9.

Częstość trafień. Na CPU przepustowość zmienia się niewiele (Rysunek 7). Przy 1000 trafień na MiB referencja i OpenMP są w obu trybach o 4-6% wolniejsze niż przy kilku trafieniach na MiB, a MPI o 14% w trybie zliczania i o 19% w trybie listy. Trafienia są rzadkie w porównaniu z liczbą bajtów: 251 417 trafień na 256 MiB to jedno na około 1000 bajtów. Obsługa trafienia to w trybie zliczania kilka dodawań, a w trybie listy dopisanie pary do lokalnego wektora. Scalenie list z dziesięciu wątków trwa poniżej milisekundy. Spadek wynika więc głównie z tego, że w miejscach trafień automat odwiedza głębokie, rzadko używane stany. MPI traci prawdopodobnie więcej, bo każdy z ośmiu procesów ma własną kopię automatu i te stany trafiają do pamięci podręcznej osobno dla każdej kopii. Wyraźnie szybszy jest tylko punkt 0, ale nie z powodu braku trafień. W tych danych automat nigdy nie opuszcza korzenia, więc cały skan odbywa się w pamięci L1.

Na GPU koszt trafień jest wyraźniejszy. Kernel zwalnia z 28,8 GB/s przy jednym trafieniu na MiB do 21,2 GB/s przy 1000, bo wątek trafiający na wzorzec przechodzi całą jego długość, a reszta warpu na niego czeka. Tryb listy wymaga drugiego przebiegu kernela i skopiowania wyników, czyli 4 MB dla 251 417 par, co obniża przepustowość w czasie pełnym z 4,8 do 3,8 GB/s. Przy takiej liczbie trafień tryb listy jest więc praktyczny na wszystkich urządzeniach, a użycie trybu zliczania w pozostałych eksperymentach nie zmienia wniosków.



Rysunek 6: Przepustowość skanu w funkcji zakresu długości wzorców, dla 256 MiB tekstu i 1000 wzorców.



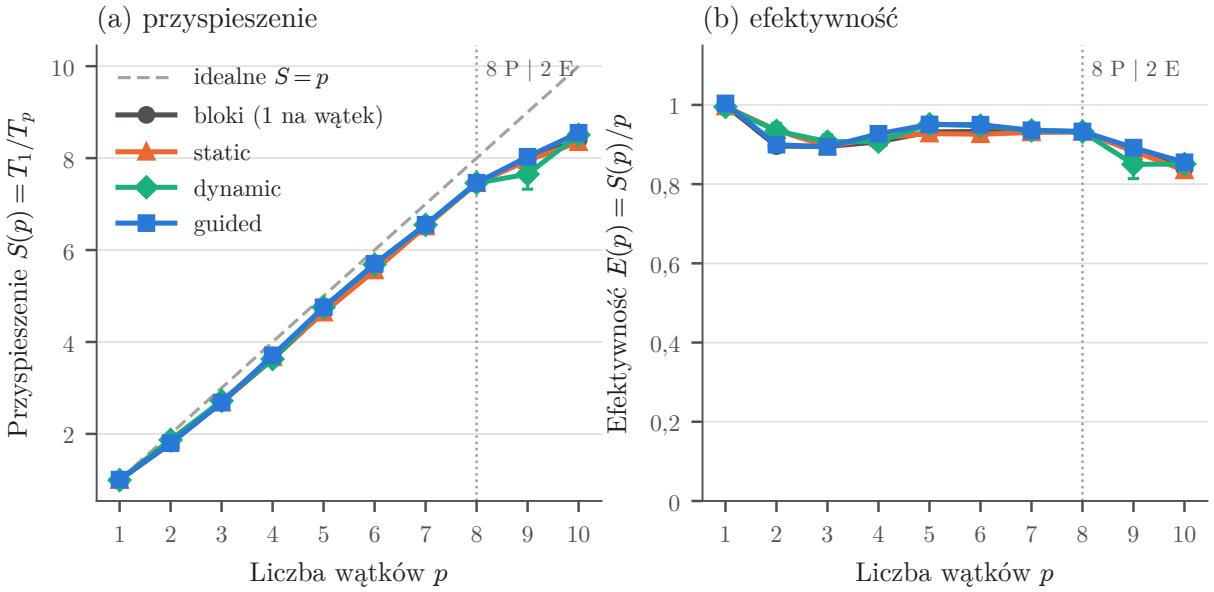
Rysunek 7: Przepustowość w czasie pełnym, bez odczytu pliku, w funkcji liczby trafień na MiB tekstu, w trybie zliczania i pełnej listy. Tekst ma 256 MiB, wzorców jest 1000, a liczba trafień rośnie od 0 do 251 417. Punkt 0 to dane z trybu `--no-match`, w których tło składa się z bajtów mniejszych od `0x80`, a wzorce zaczynają się bajtem co najmniej `0x80`.

7.4 OpenMP: przyspieszenie i efektywność

Referencja skanuje 512 MiB w 2,33 s, czyli z przepustowością 0,23 GB/s. Budowa automatu dla 1000 wzorców trwa 1,5 ms, a scalanie wyników wątków kilka mikrosekund. W pętli skanu nie ma żadnej synchronizacji, a każdy wątek zbiera wyniki we własnej strukturze, więc jedynym kosztem równoległości jest zakładka. Dzięki wczesnemu zakończeniu skanu zakładki (Sekcja 2.3) przy kawałkach 1 MiB ponownie skanowany jest średnio jeden bajt na granicę kawałka, co daje 515 bajtów na cały tekst, a nie $L_{\max} - 1 = 31$ bajtów na granicę.

Na rdzeniach wydajnych wszystkie sposoby podziału skalują się prawie liniowo (Rysunek 8). Przy ośmiu wątkach przyspieszenie wynosi 7,46, a efektywność 0,93, i dla czterech harmonogramów różni się o mniej niż 0,2%. Efektywność spada do 0,90-0,94 już przy dwóch wątkach i potem prawie się nie zmienia. Pojedynczy wątek pracuje z najwyższym zegarem rdzenia, a gdy w klastrze pracuje więcej rdzeni, zegar klastra jest niższy, a rdzenie dzielą jego pamięć L2. To najpewniej obniża efektywność już przy dwóch wątkach.

Rdzenie energooszczędne pomagają każdemu sposobowi podziału. Przy dziesięciu wątkach przyspieszenie wynosi od 8,35 dla `static` do 8,55 dla `guided`, a `dynamic` osiąga 8,51 i 1,96 GB/s. Wyniki dobrze opisuje prosty model, w którym wątek na rdzeniu wydajnym daje 0,93 prędkości referencji, a na energooszczędnym około 0,5. Model przewiduje $S(9) = 7{,}9$ i $S(10) = 8{,}4$, a zmierzone wartości to od 7,7 do 8,0 oraz od 8,35 do 8,55. Nawet równe bloki prawie nie tracą, chociaż wątki na rdzeniach energooszczędnych dostają tyle samo pracy co pozostałe. Planista macOS przenosi bowiem wątki między rdzeniami wydajnymi i energooszczędnymi, więc każdy wątek pracuje średnio z prędkością wszystkich rdzeni. Na macOS nie da się przypiąć wątku do rdzenia, dlatego nie mogliśmy sprawdzić podziału statycznego przy stałym przydziale wątków do rdzeni. Przewagę przydziału dynamicznego przy nierównym koszcie danych pokazuje Sekcja 8.

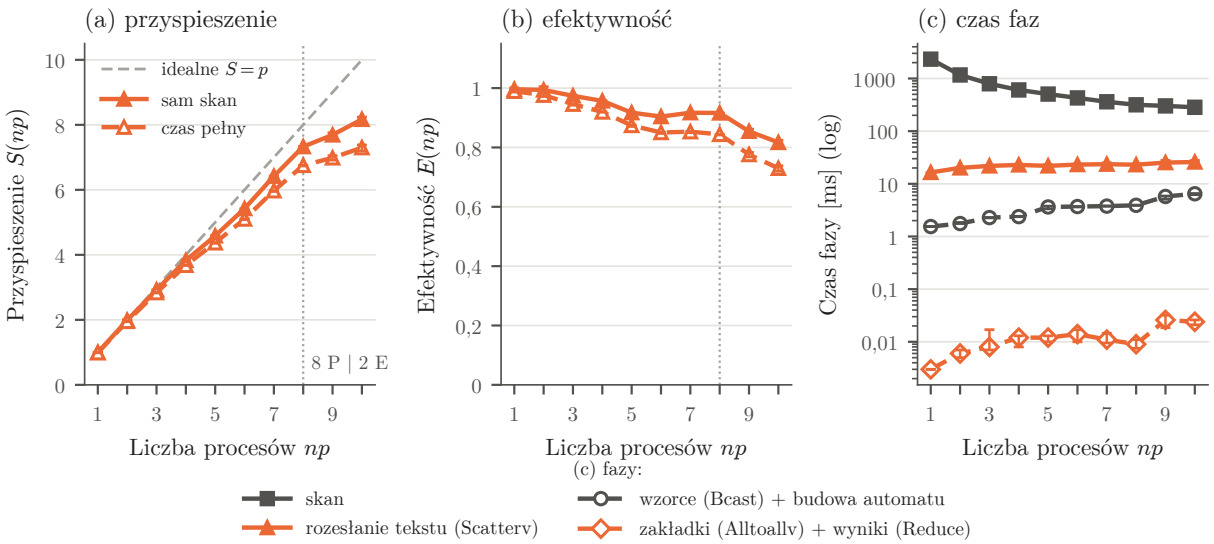


Rysunek 8: OpenMP na M1 Pro: przyspieszenie $S(p)$ i efektywność $E(p)$ względem referencji sekwencyjnej, liczone z czasu samego skanu 512 MiB tekstu przy 1000 wzorców. Słupki pokazują rozstęp międzykwartylowy czasu wersji równoległej. W harmonogramach `static`, `dynamic` i `guided` kawałki mają 1 MiB. Pionowa linia oddziela osiem rdzeni wydajnych od dwóch energooszczędnych.

7.5 MPI lokalne

Sam skan w MPI skaluje się prawie tak samo jak OpenMP z podziałem na bloki (Rysunek 9). Przyspieszenie wynosi 7,3 dla ośmiu procesów, 7,7 dla dziewięciu i 8,2 dla dziesięciu. Efektywność samego skanu utrzymuje się powyżej 0,9 do ośmiu procesów, a potem spada do 0,82 przy dziesięciu, z tych samych powodów co w OpenMP. Dla czasu pełnego wynosi 0,84 przy ośmiu i 0,73 przy dziesięciu procesach.

Różnica między krzywymi to koszt komunikacji. Rozesłanie 512 MiB tekstu przez `MPI_Scatterv` trwa od 16 do 26 ms i rośnie z liczbą procesów. Przy skanie trwającym około 320 ms obniża to przyspieszenie czasu pełnego do 6,8 dla ośmiu procesów. Pozostałe fazy są tanie. Rozesłanie 1000 wzorców i budowa automatu zajmują od 1,5 do 6 ms, przy czym czas najwolniejszego procesu rośnie z liczbą procesów. Wymiana zakładek i redukcja dwóch liczb trwają najwyżej kilka setnych milisekundy. Na jednej maszynie MPI jest więc tylko trochę wolniejsze od OpenMP, ale musi skopiować cały tekst, czego OpenMP z pamięcią wspólną robić nie musi. W prawdziwym klastrze rozesłanie tekstu przez sieć byłoby kosztem dominującym, czego nasz pomiar nie obejmuje.



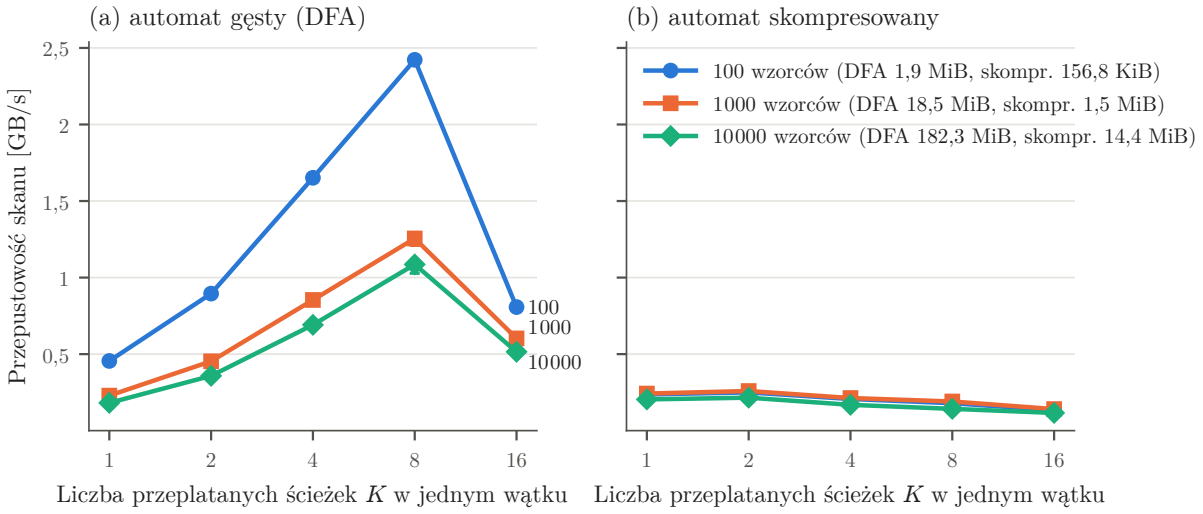
Rysunek 9: MPI uruchomione na jednym komputerze, M1 Pro z Open MPI 5. Procesy komunikują się przez pamięć współdzieloną, więc jest to wynik lokalny, a nie pomiar klastra. Panele (a) i (b) pokazują przyspieszenie i efektywność samego skanu oraz czasu pełnego, obejmującego rozesłanie wzorców i tekstu, wymianę zakładek, budowę automatu, skan i zbieranie wyników. Panel (c) pokazuje czas poszczególnych faz w skali logarytmicznej: komunikację na pomarańczowo, a obliczenia lokalne na szaro. Tekst ma 512 MiB, wzorców jest 1000.

7.6 Przeplot ścieżek: ile da się wycisnąć z jednego rdzenia

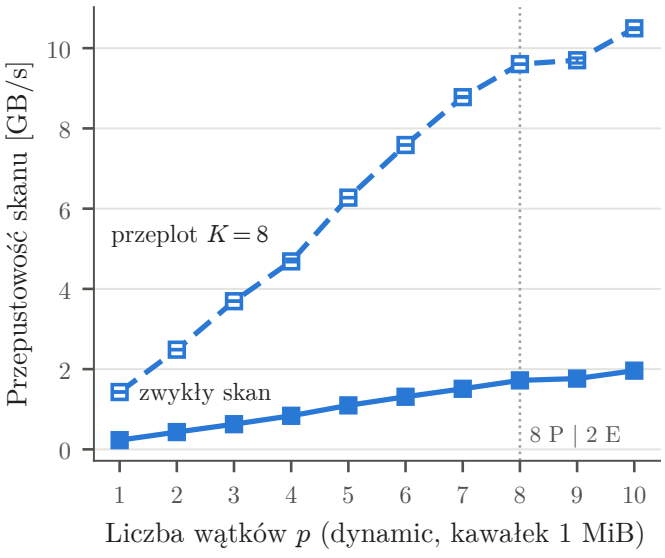
Zwykła pętla skanu wykonuje jedno przejście automatu na bajt, ale przepustowość 0,23 GB/s na rdzeniu taktowanym 3,2 GHz oznacza około 14 taktów na bajt. Tyle mniej więcej trwa odczyt z pamięci podręcznej L2, na który czeka następny krok. Przeplot K niezależnych ścieżek, opisany w Sekcja 4.3, pozwala procesorowi obsługiwać K odczytów naraz, a efekt jest duży (Rysunek 10).

- Dla gęstego DFA przepustowość rośnie prawie liniowo z K aż do ośmiu ścieżek: 5,5 raza dla 1000 wzorców, do 1,26 GB/s, 5,3 raza dla 100 wzorców, do 2,42 GB/s, i 6,0 raza dla 10 000 wzorców.
- Przy szesnastu ścieżkach wynik spada o ponad połowę. Szesnaście stanów nie mieści się w rejestrach razem z adresami pasów, a liczba chybień w pamięci podręcznej, które procesor obsługuje naraz, jest ograniczona.
- Układ skompresowany zyskuje najwyżej 7% przy dwóch ścieżkach, a od czterech traci. Jego przejście to pętla po linkach porażki z rozgałęzieniami zależnymi od danych, więc ogranicza go błędne przewidywanie skoków, a nie opóźnienie pamięci. Bez przeplotu jest porównywalny z gęstym: szybszy, gdy DFA nie mieści się w L2, i dwa razy wolniejszy dla 100 wzorców, gdy DFA ma 1,9 MiB. Z przeplotem $K = 8$ jest od 6,6 do 13,5 raza wolniejszy.

Przeplot można łączyć z wątkami. OpenMP z $K = 8$ i kawałkami 1 MiB osiąga 1,43 GB/s na jednym wątku, 9,6 GB/s na ośmiu rdzeniach wydajnych i 10,5 GB/s na wszystkich dziesięciu, czyli **45 razy więcej niż referencja** (Rysunek 11). Przyspieszenie z samych wątków jest wtedy niższe, 6,7 dla ośmiu wątków zamiast 7,5, bo 64 jednoczesne odczyty zaczynają wysycać hierarchię pamięci, a rdzenie energooszczędne dodają już tylko 9%, a nie 14%. Wynika z tego, że zwykłe OpenMP przyspiesza głównie dlatego, że każdy kolejny rdzeń czeka na pamięć niezależnie od pozostałych. Jeden rdzeń z przeplotem osiąga ten sam efekt taniej.



Rysunek 10: Przepustowość jednego wątku w funkcji liczby przeplatanych ścieżek automatu K dla 256 MiB tekstu. Legenda podaje rozmiar automatu w obu układach. Pamięć L2 klastra rdzeni wydajnych M1 Pro ma 12 MiB.



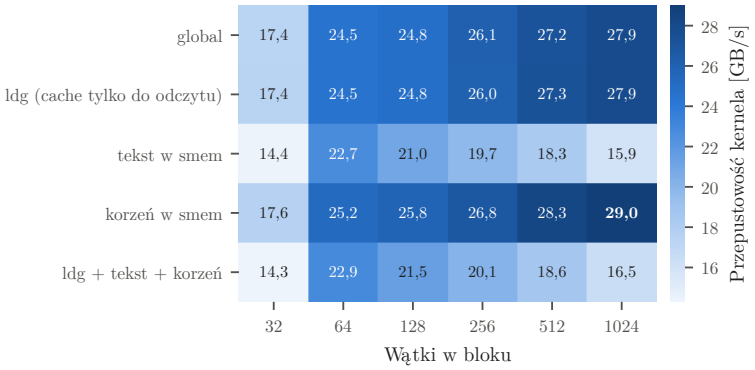
Rysunek 11: OpenMP z harmonogramem `dynamic` i kawałkami 1 MiB, bez przeplotu i z przeplotem $K = 8$, dla 512 MiB tekstu i 1000 wzorców.

7.7 GPU: PFAC na karcie Tesla T4

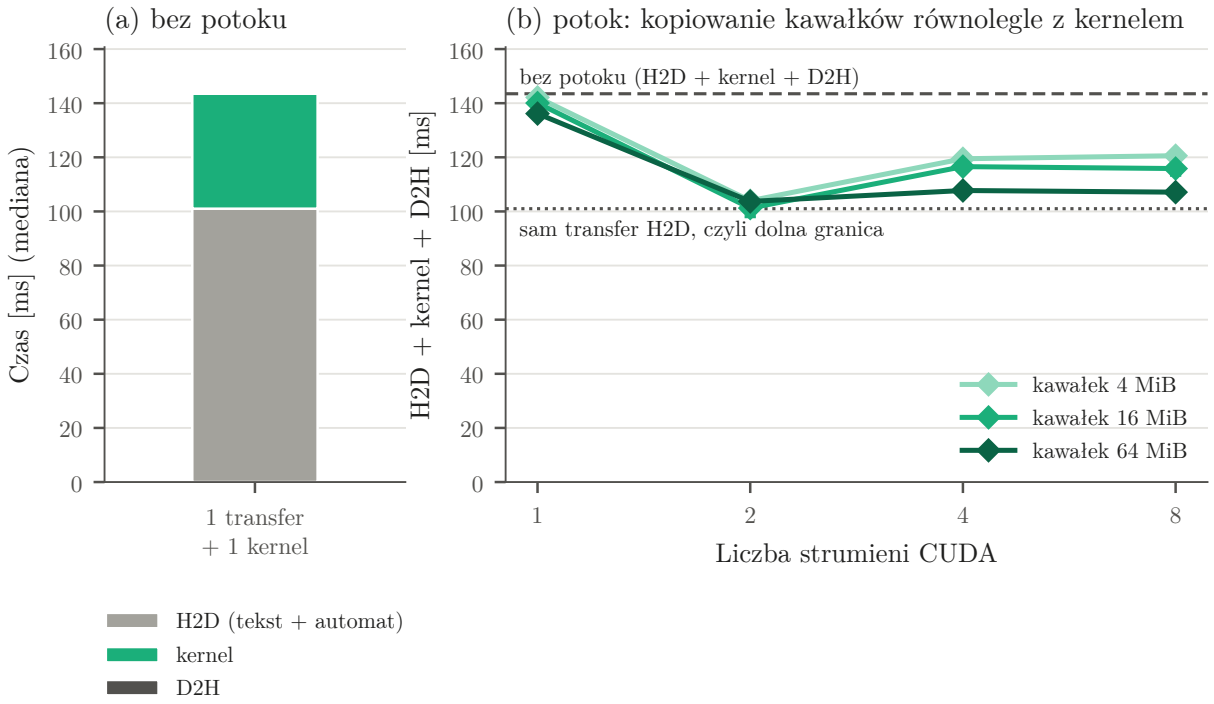
Uwaga o zegarach GPU. W pierwszej serii pomiarów czas kernela miał dwie wyraźne wartości: w jednym uruchomieniu kilka pierwszych powtórzeń trwało około 19 ms, a kolejne 9,5 ms. Odczyt z narzędzia `nvidia-smi` wyjaśnił przyczynę. Bezczynna karta jest w stanie oszczędzania energii P8 z zegarem 300 MHz i potrzebuje od pół sekundy do sekundy pracy, żeby podnieść zegar, a jedno powtórzenie rozgrzewkowe trwa krócej. Dodaliśmy więc rozgrzewkę liczoną w czasie: przed pomiarem kernel pracuje przez dwie sekundy, co ustawia opcja `--warmup-ms`. Wszystkie serie GPU powtórzyliśmy z tą rozgrzewką i rozrzut powtórzeń spadł do 1-4% dla tekstów od 256 MiB. Pierwotne pomiary zostawiliśmy w repozytorium w katalogu `results/raw/archive-gpu-bez-rozgrzewki`, ale nie używamy ich w raporcie.

Osobny pomiar zegarów pokazał drugie zjawisko. Przy ciągłej pracy kernela karta dochodzi do limitu mocy 70 W i obniża zegar do 1050-1200 MHz, a `nvidia-smi` podaje jako przyczynę właśnie limit mocy. W przerwach na kopiowanie tekstu zegar wraca do około 1575 MHz, więc krótki kernel zaczyna pracę szybciej, niż pozwala stan ustalony. Dlatego kernel zwalnia dla dużych tekstów. Dla 256 MiB trwa 9,5 ms, a dla 2 GiB 100 ms i prawie cały ten czas pracuje na obniżonym zegarze. Po zablokowaniu zegara na 1005 MHz poleceniem `nvidia-smi -lgc` czas kernela na bajt tekstu różni się między 256 MiB a 2 GiB już tylko o 7%, a nie o 32%. Zmniejszenie siatki do bloków mieszczących się naraz na karcie tej różnicy nie usuwa, więc nie wynika ona z kolejności dostępu do tekstu. Podane niżej przepustowości kernela dla tekstów do 512 MiB opisują więc krótką pracę po przerwie, a przy ciągłym skanowaniu danych już obecnych na karcie należy oczekiwać około 21 GB/s, jak dla 2 GiB. Pomiar odtwarza skrypt `bench/colab/zegary.sh`, a jego wyniki są w katalogu `results/raw/gpu-zegary`.

Warianty kernela i rozmiar bloku. Najszybszy jest wariant z wierszem korzenia w pamięci współdzielonej i blokiem 1024 wątków, który osiąga 29,0 GB/s (Rysunek 12). Pierwszy krok wykonuje każdy wątek, a na losowym tle większość wątków na nim kończy, więc kilobajtowy wiersz korzenia jest najczęściej czytanym fragmentem tablicy. Wariant `ldg` nie różni się od `global`, bo przy wskaźnikach `const __restrict__` kompilator i tak kieruje odczyty tablicy przez pamięć podręczną tylko do odczytu. Kopiowanie tekstu do pamięci współdzielonej spowalnia kernel do 15,9-22,7 GB/s, tym bardziej, im większy blok. Odczyty tekstu z pamięci globalnej i tak są łączone w duże transakcje, bo sąsiednie wątki czytają sąsiednie bajty, a wątek używa średnio jednego lub dwóch bajtów. Kopiowanie kafla i dwie bariery na kafel tylko dokładają pracy. Bloki 32-wątkowe są najwolniejsze, z wynikiem 17 GB/s, bo T4 utrzymuje na multiprocesorze najwyżej 16 bloków, co przy 32 wątkach daje połowę możliwego obciążenia. Pozostałe serie używają wariantu `smem-root` z blokiem 256 wątków, który osiąga 26,8 GB/s, o 8% mniej od



Rysunek 12: Przepustowość samego kernela PFAC, gdy dane i automat są już na karcie, w zależności od wariantu kernela i liczby wątków w bloku. Tesla T4, 512 MiB tekstu, 1000 wzorców. Najlepszy wynik pogrubiono.



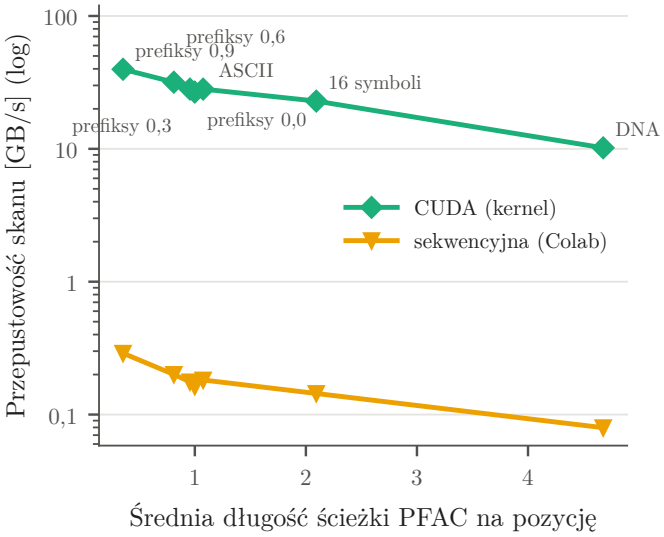
Rysunek 13: Transfer i obliczenia na GPU dla 1 GiB tekstu. Panel (a) pokazuje jeden transfer i jeden kernel, panel (b) tryb potokowy, w którym kawałki z zakładką są kopiowane asynchronicznie z pamięci przypiętej i skanowane w k strumieniach. Linia kropkowana oznacza czas samego kopiowania tekstu i automatu na kartę, czyli dolną granicę dla każdego potoku.

najlepszego. Ta różnica dotyczy tylko czasu kernela, a czas pełny i tak ogranicza łącze PCIe.

Rozmiar tekstu. Dla tekstów od 16 MiB do 1 GiB kernel osiąga od 25 do 28,5 GB/s, a dla 2 GiB 21,9 GB/s, co wynika z opisanego wyżej limitu mocy. Czas pełny jest jednak kilka razy dłuższy. Dla 512 MiB przesłanie tekstu i automatu przez PCIe 3.0 trwa 52 ms, z czego sam tekst zajmuje 48 ms, co odpowiada 11,2 GB/s. Budowa automatu na procesorze Colaba trwa 15 ms, a sam kernel 20 ms. Daje to 6,1 GB/s w czasie pełnym (Rysunek 3). Dla małych tekstów dominuje stały koszt budowy automatu i przesłania tablicy PFAC o rozmiarze 17,8 MiB, przez co dla 16 MiB czas pełny daje tylko 0,9 GB/s.

Potok strumieni. Bez potoku przetworzenie 1 GiB tekstu wymaga 101 ms kopiowania tekstu i automatu oraz 43 ms pracy kernela, razem 144 ms (Rysunek 13). Już przy dwóch strumieniach kernel liczy jeden kawałek w czasie, gdy kopiowany jest następny, i całość trwa od 101 do 104 ms, czyli tyle, ile sam transfer. Kernel jest więc całkowicie ukryty, a przyspieszenie wynosi 1,4 raza. Więcej strumieni nie pomaga, a przy małych kawałkach nawet lekko szkodzi, bo kopie i tak przechodzą kolejno przez to samo łącze PCIe, a każdy kawałek to dodatkowe wywołania sterownika. Rozmiar kawałka od 4 do 64 MiB ma niewielki wpływ. Skoro kernel przetwarza 25 GB/s, a PCIe przesyła około 11 GB/s, czas pełny ogranicza wyłącznie transfer.

Rozmiar struktury wzorców. Przepustowość kernela maleje z liczbą wzorców (Rysunek 4): 63 GB/s dla 10, 41 GB/s dla 100, 28 GB/s dla 1000, 16 GB/s dla 10 000 i 3,1 GB/s dla 100 000 wzorców. Działają tu jednocześnie dwa mechanizmy. Po pierwsze, tablica PFAC rośnie z 195 KiB do 1,7 GiB i powyżej 100 wzorców przestaje mieścić się w czteromegabajtowej pamięci L2 karty, więc przejścia stają się losowymi odczytami z pamięci GDDR6. Po drugie, rośnie praca wątku. Średnia długość ścieżki PFAC na pozycję rośnie od 0,035 kroku dla 10 wzorców do 1,66 kroku dla 100 000, bo przy wielu wzorcach prawie każdy bajt ma krawędź wychodzącą z korzenia. Dla



Rysunek 14: Wpływ częściowych dopasowań na GPU i CPU: przepustowość w funkcji średniej długości ścieżki PFAC na pozycję startową, dla 256 MiB tekstu i 1000 wzorców o długości 8-32 B. Każdy punkt to zbiór danych z innym udziałem wzorców o wspólnym prefiksie albo z innym alfabetem. Pozostałe parametry są bazowe, a oba urządzenia pracują w tym samym komputerze Colab.

100 000 wzorców czas pełny, 0,11 GB/s, wyznaczają budowa automatu na CPU, trwająca 1,8 s, i przesłanie tablicy, trwające 421 ms. W takim przypadku GPU opłaca się tylko wtedy, gdy ten sam automat służy do wielu skanów, na przykład dla strumienia wielu plików.

Częściowe dopasowania i rozbieżność wątków. Średnią długość ścieżki PFAC na pozycję, czyli pracę jednego wątku, zmienialiśmy dwoma parametrami generatora (Rysunek 14). Im więcej wzorców ma wspólny czterobajtowy prefiks, tym krótsze są ścieżki: 0,96 kroku przy 30% takich wzorców i 0,35 przy 90%. Wspólne prefiksy zajmują tylko kilka z 256 krawędzi korzenia, więc więcej wątków kończy pracę już na pierwszym bajcie. Mniejszy alfabet działa odwrotnie, bo losowe tło częściej powtarza początki wzorców. Alfabet szesnastoznakowy daje 2,1 kroku, a czteroznakowy alfabet DNA 4,7 kroku. Przepustowość kernela maleje razem z długością ścieżki: od 39,7 GB/s przy 0,35 kroku, przez 28,2 GB/s dla danych bazowych, do 10,2 GB/s dla DNA. Spadek jest mniejszy niż przyrost pracy, bo część kosztu jest stała: odczyt tekstu i pierwszy krok z wiersza korzenia w pamięci współdzielonej.

Ten sam kierunek widać na CPU, gdzie przepustowość spada z 0,29 do 0,08 GB/s. Aho-Corasick wykonuje wprawdzie zawsze jedno przejście na bajt, ale przy częstych częściowych dopasowaniach odwiedza głębsze, rzadziej używane stany i częściej chybia w pamięci podręcznej. Stosunek wydajności GPU do CPU jest więc prawie stały i wynosi od 137 do 164 razy dla wszystkich zbiorów z wyjątkiem DNA, gdzie spada do 128. Tam rozbieżność wątków w warpach, w których jeden wątek idzie głęboko, a reszta czeka, kosztuje GPU nieco więcej niż CPU. Wpływ długości wzorców i gęstości częściowych dopasowań na wybór urządzenia badamy dokładniej w sekcji 9.

Przyspieszenie względem jawnie wskazanej referencji. Dla danych bazowych o rozmiarze 512 MiB kernel T4 osiąga 27,1 GB/s. Jest to 162 razy więcej niż referencja sekwencyjna uruchomiona na tym samym komputerze Colab, która osiąga 0,17 GB/s, i 36 razy więcej niż najlepsza wersja CPU na Colabie, czyli przeplot $K = 8$ z wynikiem 0,76 GB/s. W czasie pełnym przewaga nad referencją Colab spada do 37 razy przy jednym transferze, co daje 6,1 GB/s, i do 46 razy w trybie potokowym, z wynikiem 7,7 GB/s. Najlepsza wersja CPU na M1 Pro, czyli OpenMP z dziesięcioma wątkami i przeplotem, osiąga w czasie pełnym 10,2 GB/s. GPU z transferami jest więc od niej wolniejsze i osiąga 0,76 jej wyniku, a przewagę, 2,6 raza, ma tylko sam kernel.

Tabela 5: Zestawienie wersji dla danych bazowych: 512 MiB tekstu i 1000 wzorców, mediana z siedmiu powtórzeń. Kolumny „skan” dotyczą samego skanu lub samego kernela, kolumny „pełny” czasu z budową automatu, komunikacją lub transferami, bez odczytu pliku. Przyspieszenie liczymy względem referencji sekwencyjnej na tej samej maszynie, czyli M1 Pro albo Colab. Wiersze M1 Pro pochodzą z serii przyspieszenia (Rysunki 8 i 9), a wiersze Colab z serii rozmiaru tekstu (Rysunek 3). W trybie potokowym czas kernela obejmuje nakładające się transfery, więc podajemy tylko czas pełny. Tabela podaje potok z czterema strumieniami, a według rysunku 13 dwa strumienie są o około 10% szybsze.

		Przepustowość [GB/s]		Przyspieszenie	
Wersja	Sprzęt	skan	pełny	skan	pełny
sekwencyjna	M1 Pro, 1 rdzeń	0,23	0,23	1×	1×
OpenMP, $p = 8$, bloki	M1 Pro, 8 P	1,72	1,71	7,5×	7,4×
OpenMP, $p = 10$, dynamic 1 MiB	M1 Pro, 8 P + 2 E	1,96	1,95	8,5×	8,5×
OpenMP, $p = 10$ + przeplot $K = 8$	M1 Pro, 8 P + 2 E	10,49	10,17	45,5×	44,1×
MPI, $np = 8$	M1 Pro, 8 P	1,70	1,56	7,3×	6,8×
sekwencyjna	Colab, Xeon 2,0 GHz	0,17	0,17	1×	1×
sekwencyjna + przeplot $K = 8$	Colab, Xeon 2,0 GHz	0,76	0,75	4,5×	4,5×
OpenMP, $p = 2$	Colab, 2 vCPU	0,28	0,28	1,7×	1,6×
OpenMP, $p = 2$ + przeplot $K = 8$	Colab, 2 vCPU	0,73	0,73	4,4×	4,3×
CUDA PFAC, 1 transfer	Tesla T4	27,12	6,14	161,7×	36,7×
CUDA PFAC, potok 4 strumienie	Tesla T4	n/d	7,73	n/d	46,2×

Wszystkie wersje przy tym samym zakresie mierzonych operacji zestawia Tabela 5.

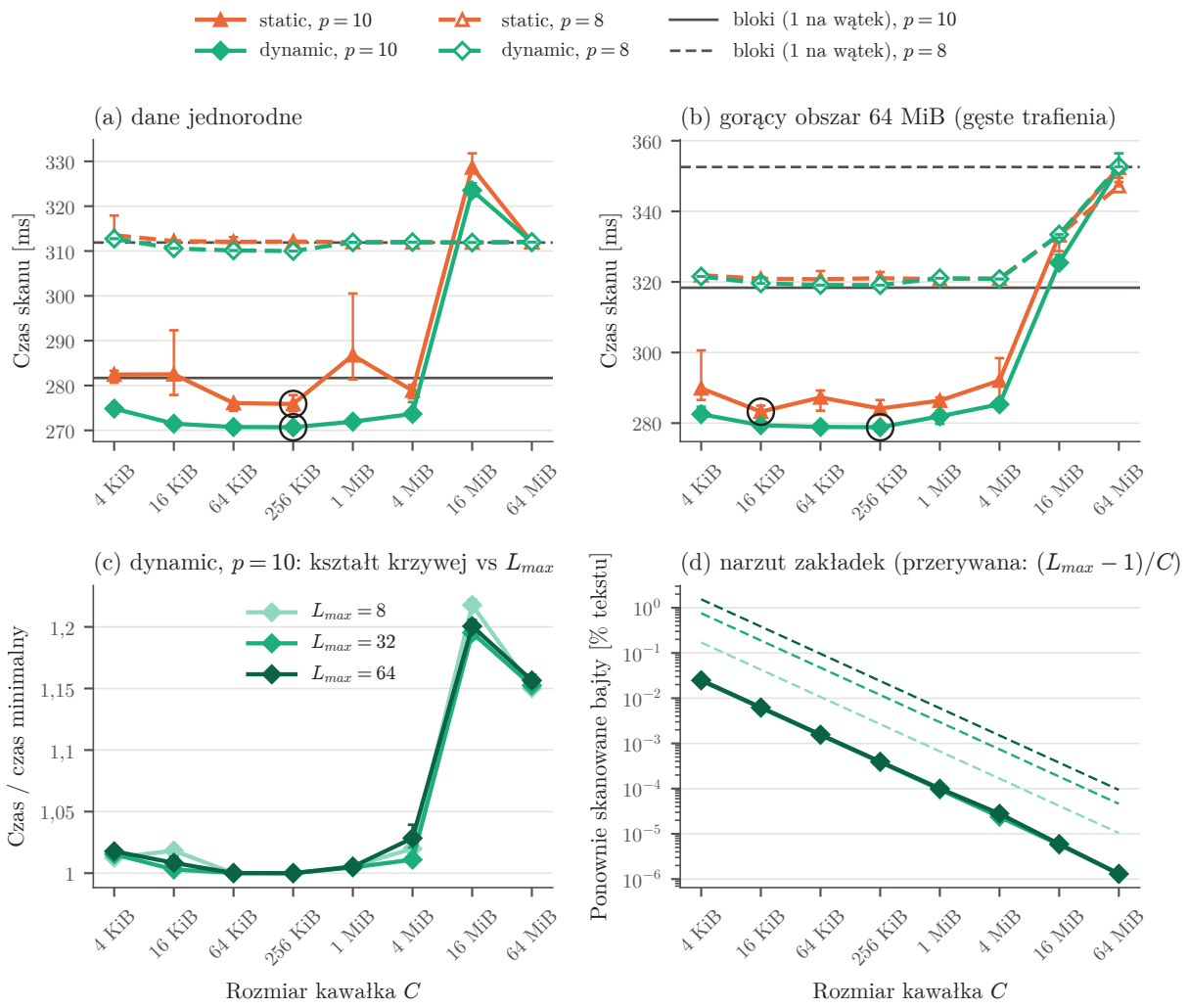
8 Dodatek dla pary: podział tekstu i krzywa U

Od zespołu dwuosobowego karta tematu wymaga porównania dwóch sposobów podziału tekstu: kilku dużych bloków przydzielanych statycznie i wielu małych kawałków przydzielanych dynamicznie, razem z wykresem czasu w funkcji rozmiaru kawałka C. Spodziewamy się krzywej w kształcie litery U.

- **Lewe ramię, małe kawałki.** Każdy kawałek kosztuje stałą porcję pracy: pobranie numeru kawałka z harmonogramu, co przy `dynamic` oznacza operację atomową na wspólnym liczniku, start skanu w korzeniu i ponowne przetworzenie zakładki. Narzut zakładki wynosi w najgorszym razie $(L_{\max} - 1)/C$ długości tekstu.
- **Prawe ramię, duże kawałki.** Kawałków jest za mało, żeby wyrównać różnice prędkości wątków, wynikające z rdzeni o różnej wydajności i z przerw, gdy system zabiera wątkowi rdzeń, oraz różny koszt fragmentów tekstu. Gdy $C > n/p$, część wątków w ogóle nie dostaje pracy.
- **Dno.** Kawałków jest dość, żeby zrównoważyć obciążenie, a narzut na kawałek jest pomijalny.

Mierzymy OpenMP z ośmioma wątkami, czyli tylko na rdzeniach wydajnych, i z dziesięcioma, czyli na wszystkich rdzeniach. Sprawdzamy harmonogramy `static`, który przydziela kawałki cyklicznie po jednym, i `dynamic`, dla kawałków od 4 KiB do 64 MiB. Punktem odniesienia jest podział na równe bloki, po jednym na wątek. Używamy czterech zbiorów danych po 512 MiB:

1. jednorodnych danych bazowych;
2. danych z **gorącym obszarem**, w których 64 MiB w środku tekstu, od 224 do 288 MiB, zawiera prawie wszystkie z 614 000 wstawionych trafień i częściowych dopasowań. Skan tego obszaru jest około 1,25 raza wolniejszy, bo gęste trafienia prowadzą automat do głębokich, rzadko odwiedzanych stanów;



Rysunek 15: Krzywa U: czas skanu OpenMP w funkcji rozmiaru kawałka C na M1 Pro, dla 512 MiB tekstu i 1000 wzorców. Poziome linie oznaczają równe bloki, po jednym na wątek, a kółka minimum krzywej dla dziesięciu wątków. Panel (c) pokazuje krzywe dla harmonogramu `dynamic` i dziesięciu wątków przy trzech zakresach długości wzorców, podzielone przez własne minimum. Panel (d) pokazuje zmierzony odsetek bajtów skanowanych ponownie w zakładkach, gdzie linie ciągłe trzech zbiorów się pokrywają, oraz ograniczenie teoretyczne $(L_{\max} - 1)/C$ liniami przerywanymi.

3. oraz dwóch zbiorów ze wzorcami o długości 4-8 B i 32-64 B, czyli z innym $L_{\max}$ i inną długością zakładki.

Wyniki. Wszystkie opisane niżej zależności widać na rysunku 15.

- **Dane jednorodne, dziesięć wątków.** Harmonogram `dynamic` tworzy wyraźną literę U. Czas wynosi 275 ms dla kawałków 4 KiB, osiąga minimum **271 ms dla kawałków od 64 do** 256 KiB i rośnie do 324 ms dla 16 MiB. Na dziesięć wątków przypadają wtedy 32 kawałki, więc w ostatniej rundzie pracują tylko dwa wątki. Dla 64 MiB czas spada do 312 ms, bo osiem kawałków trafia do ośmiu wątków, a dwa pozostałe nic nie robią. Odpowiada to pracy na samych rdzeniach wydajnych. Równe bloki trwają 282 ms, czyli o 4% dłużej niż optimum. Harmonogram `static` z kawałkami do 4 MiB daje od 276 do 287 ms i ma większy rozrzut niż `dynamic`.
- **Dane jednorodne, osiem wątków**, czyli tylko rdzenie wydajne. Wszystkie strategie dają od 310 do 314 ms, a krzywa jest płaska, z różnicami poniżej 1%. Na jednakowych rdzeniach

i przy równym koszcie danych nie ma czego równoważyć, więc przydział dynamiczny nic nie zyskuje, ale też nic nie kosztuje.

- **Gorący obszar.** Tutaj różnice są największe. Równe bloki trwają **318 ms** przy dziesięciu wątkach i 353 ms przy ośmiu, o 13% dłużej niż na danych jednorodnych. Gorący obszar wypada na granicy bloków przy 256 MiB, więc dzielą go dwa wątki i oba pracują dłużej od pozostałych. Harmonogram `dynamic` z kawałkami od 16 KiB do 1 MiB rozdziela gorący obszar między wszystkie wątki i osiąga **279 ms**, czyli jest **1,14 raza szybszy** od bloków przy dziesięciu wątkach i 1,10 raza przy ośmiu. Czas rośnie wtedy tylko o 3% względem danych jednorodnych, co odpowiada dodatkowej pracy w gorącym obszarze. Prawe ramię jest strome: dla kawałków 64 MiB gorący obszar trafia do jednego kawałka i czas rośnie do 352 ms. Także `static` ma tu kształt litery U, z minimum 283 ms dla kawałków 16 KiB.
- **Lewe ramię jest płytkie** i nie wynika z zakładek. Dla kawałków 4 KiB czas rośnie tylko o 1-2% względem minimum. Dzięki wczesnemu zakończeniu skanu zakładki ponownie przetwarzamy średnio **jeden bajt na granicę kawałka, niezależnie od** $L_{\max}$. Dla kawałków 4 KiB jest to 132 759 bajtów na 131 072 kawałki, czyli 0,025% tekstu, podczas gdy ograniczenie $(L_{\max} - 1)/C$ wynosi od 0,17 do 1,5%. Lewe ramię wynika więc z kosztu pobrania kawałka z harmonogramu i ze startu automatu w korzeniu, gdy pierwsze przejścia kawałka trafiają w zimną pamięć podręczną. Dlatego optimum nie zależy od $L_{\max}$: dla wszystkich trzech długości wzorców wypada przy kawałkach od 64 do 256 KiB, co pokazuje panel (c).

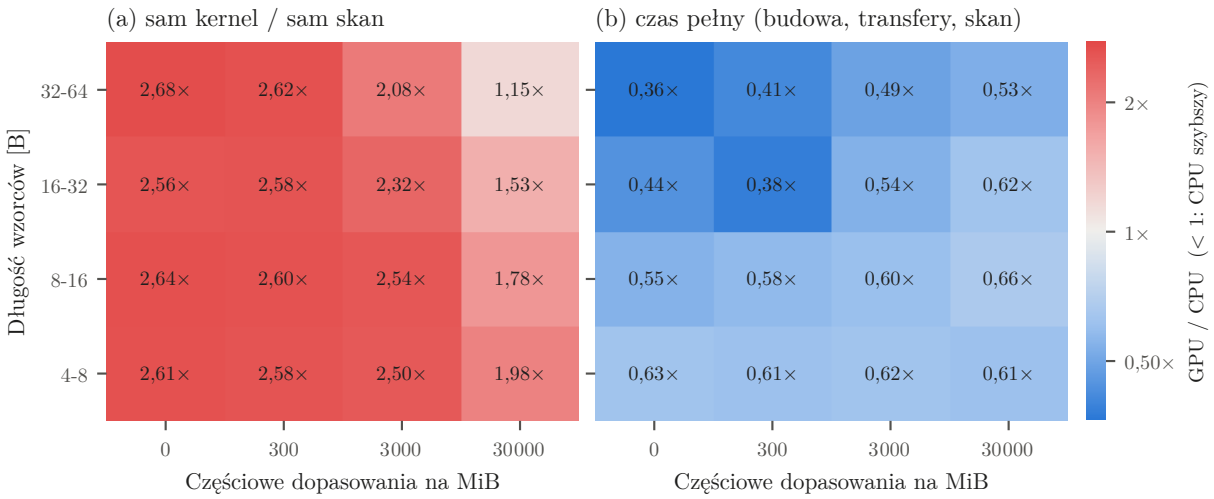
Wniosek dla pary. Gdy koszt danych jest równy, a rdzenie jednakowe, podział na p bloków jest optymalny i najprostszy. Z rdzeniami energooszczędnymi bloki tracą 4%, a przy nierównym koszcie fragmentów tekstu, na przykład przy gęstych trafieniach lub długich częściowych dopasowaniach, typowych dla ruchu zawierającego atak, od 10 do 14%. **Małe kawałki od** 16 KiB **do** 1 MiB **z przydziałem dynamicznym** dają wtedy wynik od 1,10 do 1,14 raza lepszy. Kawałek zbyt mały kosztuje przy tym 1-2%, a zbyt duży od 20 do 26%. Zakładka $L_{\max} - 1$ z wczesnym zakończeniem skanu przestaje decydować o wyborze rozmiaru kawałka.

Wyniki są powtarzalne między seriami. Równe bloki przy ośmiu wątkach trwają tutaj 312 ms, tyle samo co w seriach E1 i E5, a te same konfiguracje zmierzone w różnych seriach różnią się o najwyżej 3% (Sekcja 6.1).

9 Pytanie badawcze: CPU czy GPU?

Karta tematu proponuje pytanie: *czy długość wzorców i udział częściowych dopasowań przewidują wybór CPU lub GPU?* Z budowy obu algorytmów wynika następujące przypuszczenie. Aho-Corasick na CPU wykonuje jedno przejście automatu na bajt niezależnie od danych, a zmienia się tylko to, jak często odwiedzane stany są w pamięci podręcznej. Koszt PFAC na GPU to suma długości ścieżek wszystkich wątków. Na losowym tle większość wątków kończy po zerze lub jednym kroku, ale każde częściowe dopasowanie prowadzi jeden wątek głęboko, a cały warp czeka na ten najdłuższy. Dłuższe wzorce i więcej częściowych dopasowań powinny więc szkodzić GPU bardziej niż CPU.

Eksperyment. Przygotowaliśmy siatkę 4×4. Jedną osią jest długość wzorców: 4-8, 8-16, 16-32 i 32-64 bajty. Drugą osią jest gęstość wstawionych częściowych dopasowań: 0, 300, 3000 i 30 000 na MiB. Każdy zbiór ma 256 MiB tekstu i 1000 wzorców, z których 60% ma wspólny prefiks. Na tych samych, bajtowo identycznych danych mierzymy najlepszą wersję CPU, czyli OpenMP z dziesięcioma wątkami i przeplotem $K = 8$ na M1 Pro, oraz GPU z kernelem `smem-root` na T4. Dla obu porównujemy osobno czas samego skanu lub kernela i czas pełny bez odczytu pliku,



Rysunek 16: Stosunek przepustowości GPU, czyli karty T4 z kernelem `smem-root`, do najlepszej wersji CPU, czyli OpenMP z dziesięcioma wątkami i przeplotem $K = 8$ na M1 Pro. Siatka obejmuje długość wzorców i gęstość częściowych dopasowań, a każdy zbiór ma 256 MiB tekstu i 1000 wzorców. Obie maszyny pracują na tych samych danych. Panel (a) porównuje sam kernel z samym skanem, a panel (b) czasy pełne bez odczytu pliku. Kolor czerwony oznacza szybsze GPU, a niebieski szybszy CPU.

obejmujący budowę automatu, transfery i skan. Dla każdego zbioru liczymy też średnią długość ścieżki PFAC na pozycję startową, która mierzy pracę jednego wątku GPU.

Wyniki. Siatkę pokazuje Rysunek 16.

- **Sam kernel wobec samego skanu.** GPU jest szybsze w każdej komórce, ale jego przewaga zależy od obu parametrów. Bez częściowych dopasowań wynosi od 2,6 do 2,7 raza dla każdej długości wzorców, a przy 3000 częściowych dopasowań na MiB od 2,1 do 2,5 raza. Przy 30 000 częściowych dopasowań na MiB spada do 2,0 raza dla wzorców 4-8 B i do 1,2 raza dla wzorców 32-64 B. Kernel zwalnia wtedy z 32 do 5,5 GB/s, bo częściowe dopasowanie długiego wzorca oznacza dla jednego wątku ścieżkę długości nawet 63 kroków, na której koniec czeka cały warp. Średnia długość ścieżki rośnie przy tym tylko z 0,8 do 1,2 kroku. Liczy się więc najdłuższa ścieżka w warpie, a nie średnia. CPU też zwalnia, z 11,8 do 4,7 GB/s, bo średnia głębokość stanu automatu rośnie z 0,8 do 7,4, a głębokie stany rzadko są w pamięci podręcznej. CPU traci jednak mniej niż GPU.
- **Czas pełny.** Tutaj **CPU wygrywa w każdej komórce**, a GPU osiąga od 0,36 do 0,66 jego przepustowości. Nawet przy krótkich wzorcach około dwóch trzecich czasu pełnego GPU zajmuje przesłanie tekstu i automatu przez PCIe, czyli 25 ms na 256 MiB, podczas gdy kernel trwa tylko 8 ms. Dla wzorców 32-64 B dochodzi budowa automatu o rozmiarze 45 MiB na wolniejszym procesorze Colaba, która trwa 32 ms zamiast 4,4 ms na M1 Pro, oraz jego przesłanie.
- **Odpowiedź zależy od wybranej referencji.** W porównaniu ze zwykłym OpenMP bez przeplotu na tym samym M1 Pro GPU wygrywa w czasie pełnym w całej siatce, od 1,6 do 3,3 raza, a w porównaniu z referencją sekwencyjną od 13 do 27 razy. Takie porównanie zestawia jednak GPU ze słabszą implementacją CPU. Dlatego jako główny wynik podajemy porównanie z najlepszą wersją.

Prosty model progowy. Dla tekstu o długości n czasy pełne można przybliżyć wzorami

$$T_{\mathrm{GPU}} \approx T_{\mathrm{bud}} + \frac{n}{B_{\mathrm{PCIe}}} + \frac{n}{K(L,\pi)}, \qquad T_{\mathrm{CPU}} \approx T'_{\mathrm{bud}} + \frac{n}{C(L,\pi)},$$

gdzie B_{PCIe} wynosi około 11 GB/s, a K i C to przepustowości kernela i najlepszego skanu CPU, które maleją wraz z długością wzorców L i gęstością częściowych dopasowań π. Jeśli pominąć budowę automatu, GPU może wygrać tylko wtedy, gdy

$$C(L,\pi) < \left(\frac{1}{B_{\mathrm{PCIe}}} + \frac{1}{K(L,\pi)}\right)^{-1}.$$

Dla danych bazowych prawa strona wynosi około 8 GB/s, bo K jest bliskie 32 GB/s, a CPU z przeplotem osiąga od 11,8 do 12,5 GB/s, więc GPU przegrywa. Dla wzorców 32-64 B przy 30 000 częściowych dopasowań na MiB prawa strona spada do 3,7 GB/s, bo K wynosi 5,5 GB/s, a CPU osiąga 4,7 GB/s. Model przewiduje więc przewagę CPU około 1,3 raza, a zmierzony stosunek 0,53 jest jeszcze niższy z powodu kosztu budowy i przesłania automatu. Model wyjaśnia więc, dlaczego na tej parze urządzeń GPU nie wygrywa: *długie wzorce i częściowe dopasowania spowalniają zarówno GPU, jak i CPU, a łącze PCIe ogranicza GPU niezależnie od danych.*

Wniosek. Na pytanie z karty odpowiadamy twierdząco, ale wynik jest inny, niż podpowiada założenie, że GPU zawsze jest szybsze. Długość wzorców i udział częściowych dopasowań przewidują *wielkość przewagi obliczeniowej* GPU, która maleje z 2,6–2,7-krotnej do 1,2-krotnej. O wyborze urządzenia decyduje jednak przede wszystkim to, czy tekst musi przejść przez PCIe. Gdy dane trzeba dopiero przesłać, dobrze napisany kod CPU z przeplotem jest na naszym sprzęcie szybszy. Gdy dane już są na karcie, na przykład przy wielokrotnym skanowaniu tego samego bufora albo gdy karta sieciowa przesyła pakiety bezpośrednio do GPU, wygrywa GPU, i to najbardziej przy krótkich wzorcach i niewielu częściowych dopasowaniach. Przy ciągłej pracy jego przewaga jest jednak o około jedną czwartą mniejsza niż w naszych pomiarach kernela, bo karta pracuje wtedy na limicie mocy (Sekcja 7.7).

Ograniczenia i dalsza praca. Porównujemy jedną parę urządzeń: kartę T4 na łączu PCIe 3.0 i laptopowy procesor M1 Pro. Konkretne progi nie przeniosą się więc na inny sprzęt, ale kształt modelu tak. Dane są syntetyczne, a częściowe dopasowania wstawiamy jako prefiksy wzorców. Pełniejsza odpowiedź wymagałaby trzech kroków. Po pierwsze, pomiarów na GPU z szybszym łączem albo z pamięcią wspólną z CPU, jak w układach NVIDIA GH200 lub w procesorach Apple z biblioteką Metal. Po drugie, prawdziwego ruchu sieciowego, na przykład ze zbioru CIC IDS2017, i prawdziwych reguł systemów Snort lub Suricata. Po trzecie, wersji PFAC, która lepiej radzi sobie z rozbieżnością wątków, oraz porównania z gotowymi bibliotekami, takimi jak Hyperscan, jako mocną referencją.

10 Podsumowanie i wnioski

Zrealizowaliśmy pełny zakres karty C02 dla zespołu dwuosobowego: referencję, trzy technologie równoległe, generator danych z zapisanymi pozycjami trafień, porównanie dwóch sposobów podziału tekstu z krzywą U oraz drugi układ automatu. Poniżej zebraliśmy najważniejsze wnioski. Przy każdym wskazujemy rysunek lub tabelę, z których wynika.

1. **Reguła właściciela z zakładką** $L_{\max} - 1$ daje dokładnie te same trafienia przy każdym podziale, od trzybajtowych kawałków, przez dziesięć procesów MPI, po kawałki kopiowane na GPU. Dzięki wczesnemu zakończeniu skanu zakładki ponownie skanowany jest średnio jeden bajt na granicę, więc narzut zakładek jest pomijalny (Sekcja 5 i Rysunek 15).

2. **OpenMP skaluje się prawie liniowo na rdzeniach wydajnych**, z efektywnością 0,93 przy ośmiu wątkach. Rdzenie energooszczędne dodają kolejne 12-15% przy każdym sposobie podziału, bo planista macOS przenosi wątki między rdzeniami (Rysunek 8).

3. **MPI na jednej maszynie** skaluje się podobnie, ale płaci za rozesłanie całego tekstu i za osobną kopię automatu w każdym procesie. Przy dużych zbiorach wzorców zmniejsza to przepustowość o połowę w porównaniu z OpenMP (Rysunki 4 i 9).

4. **Aho-Corasick na CPU ogranicza opóźnienie pamięci.** Przeplot ośmiu ścieżek automatu podnosi przepustowość jednego rdzenia od 5,3 do 6 razy, a razem z dziesięcioma wątkami daje wynik 45 razy lepszy od referencji, podczas gdy same wątki dają 8,5 raza (Rysunki 10 i 11). Przy wyborze układu automatu ważniejsze od rozmiaru okazało się to, czy przejście odbywa się bez rozgałęzień. Gęsty DFA z przeplotem wygrywa z automatem skompresowanym nawet dla 100 000 wzorców (Rysunek 5).

5. **Dodatek dla pary.** Przy jednorodnych danych i jednakowych rdzeniach równe bloki są optymalne, a z rdzeniami energooszczędnymi tracą 4%. Przy nierównym koszcie danych małe kawałki od 16 KiB do 1 MiB z przydziałem dynamicznym są do 1,14 raza szybsze, a zbyt małe kawałki kosztują tylko 1-2% (Rysunek 15).

6. **GPU.** Najlepszy wariant kernela trzyma wiersz korzenia w pamięci współdzielonej i osiąga 29 GB/s, a kopiowanie tekstu do tej pamięci szkodzi (Rysunek 12). Przepustowość kernela maleje wraz z rozmiarem tablicy PFAC i długością ścieżek wątków (Rysunki 4 i 14). Czas pełny ogranicza łącze PCIe, a potok z dwoma strumieniami całkowicie ukrywa kernel za transferem (Rysunek 13).

7. **CPU czy GPU.** Kernel T4 jest od 1,2 do 2,7 raza szybszy od najlepszej wersji CPU, ale po doliczeniu transferów przegrywa z nią w całej badanej siatce. Długie wzorce i częste częściowe dopasowania zmniejszają przewagę GPU (Rysunek 16 i Tabela 5).

8. **Metodyka.** Rozgrzewkę GPU trzeba liczyć w czasie, a nie w powtórzeniach, bo bezczynna karta T4 podnosi zegar dopiero po około sekundzie pracy. Przy dłuższej pracy karta dochodzi do limitu mocy 70 W i obniża zegar, więc krótki kernel jest o około 30% szybszy niż ciągłe skanowanie. Na Macu przy 8-10 wątkach skrypty uruchomione obok pomiaru sprawiały, że te same konfiguracje zmierzone w różnych seriach różniły się nawet o 25%, dlatego wszystkie serie CPU powtórzyliśmy bez innych obciążeń. Generator danych nie może korzystać z rozkładów biblioteki standardowej, jeśli te same dane mają powstawać na różnych systemach (Sekcje 6.1 i 7.7 i Sekcja 3).

10.1 Ograniczenia

- Pomiary CPU wykonaliśmy na jednym laptopie z rdzeniami o różnej wydajności i bez możliwości przypinania wątków, więc o przydziale wątków do rdzeni decyduje macOS. Powtarzalność na poziomie 3% między seriami uzyskaliśmy dopiero po wstrzymaniu procesów systemu działających w tle.

- MPI mierzyliśmy na jednej maszynie, gdzie procesy komunikują się przez pamięć współdzieloną. Wyniki nie opisują więc klastra, w którym przesłanie tekstu przez sieć byłoby kosztem dominującym.

- GPU to jedna karta T4 na łączu PCIe 3.0 w środowisku współdzielonym z innymi użytkownikami. Karta pracuje na limicie mocy, więc przepustowość kernela zależy od tego, jak długo trwa jego praca. Porównania CPU z GPU dotyczą tej konkretnej pary urządzeń, a nie architektur w ogóle.

- Dane są syntetyczne. Generator kontroluje cechy istotne dla kosztu dopasowania, ale nie odtwarza rozkładu bajtów prawdziwego ruchu sieciowego, na przykład ze zbioru CIC IDS2017, którego nie używaliśmy.

10.2 Podział pracy

Osoba	Zakres prac
Piotr Waluszek	automat Aho-Corasick i referencja sekwencyjna, wersja OpenMP, kernel CUDA, dodatek dla pary, pomiary na CPU, pytanie badawcze
Emilia Trybała	generator danych, wersja MPI, transfery i tryby pracy w wersji CUDA, testy poprawności, pomiary na GPU, sprawozdanie
wspólnie	metodyka pomiarów, interpretacja wyników

A Odtwarzalność

Repozytorium `https://github.com/PiotrWaluszek/prir-c02-wyszukiwanie-sygnatur` zawiera kod, testy, opisy eksperymentów, surowe wyniki ze wszystkimi powtórzeniami, opis konfiguracji obu maszyn i skrypty, które tworzą każdy wykres i każdą tabelę tego raportu. Wyniki w raporcie pochodzą z wersji oznaczonej tagiem `v1.0`.

Budowa i testy. CMake sam wykrywa dostępne OpenMP, MPI i CUDA i domyślnie buduje wersję zoptymalizowaną.

```
cmake -B build && cmake --build build -j
cd build && ctest --output-on-failure        # testy jednostkowe i test integracyjny
```

Uruchomienie na innym komputerze. Najprościej przez Docker: obraz z Ubuntu 24.04 zawiera kompilator, OpenMP i Open MPI, więc wystarczy Docker Desktop na Windows, macOS albo Linuksie.

```
docker build -t c02 .
docker run --rm c02                          # demo: wszystkie wersje CPU na tych
    samych danych
```

Projekt można też zbudować bezpośrednio na macOS (Apple clang z libomp, Open MPI) i na Windows (Visual Studio z OpenMP, MS-MPI). Skrypty `demo/demo.sh` i `demo/demo.ps1` generują dane, uruchamiają wersję sekwencyjną, OpenMP i MPI i sprawdzają, że dały ten sam wynik. Instrukcja krok po kroku jest w pliku `docs/URUCHOMIENIE.md`. Po każdej zmianie kodu GitHub Actions buduje projekt i uruchamia testy na macOS, Windows i Linuksie oraz w obrazie Dockera.

Pomiary CPU na Macu. Pliki JSON w katalogu `bench/configs/mac` opisują poszczególne eksperymenty. Skrypt `bench/run.py` generuje brakujące zbiory danych i zapamiętuje je pod skrótem parametrów, uruchamia programy w losowej kolejności, a wyniki dopisuje do pliku CSV o nazwie eksperymentu w katalogu `results/raw`. Wszystkie serie na M1 Pro trwają około 50 minut. Przed ich uruchomieniem trzeba zamknąć inne programy i najlepiej wstrzymać procesy działające w tle (Sekcja 6.1).

```
bash bench/env_info.sh                       # opis sprzętu w results/env
python3 bench/run.py bench/configs/mac/*.json
```

Pomiary GPU na Colabie. Potrzebne jest zainstalowane i zalogowane narzędzie Colab CLI [18]. Skrypt tworzy maszynę z kartą T4, wysyła na nią kod z bieżącej wersji, buduje go i uruchamia testy, a przy błędzie przerywa pracę. Następnie uruchamia kolejne serie w tle na maszynie wirtualnej i pobiera wyniki każdej serii zaraz po jej zakończeniu.

```
bench/colab/run_gpu.sh                       # wszystkie serie z bench/configs/gpu
```

```
bench/colab/remote.sh c02 < bench/colab/zegary.sh   # zegary i limit mocy karty
```

```
python3 scripts/check_results.py                # zgodność liczby trafień i sum
    kontrolnych
python3 scripts/datasets.py                     # parametry i ziarno każdego zbioru
    danych
python3 scripts/plots/make_all.py               # wszystkie rysunki i tabele raportu
```

Zbiory danych. W plikach wyników każdy zbiór ma nazwę `ds-<skrót parametrów>`. Plik `results/raw/datasets.csv` podaje dla każdego z 48 użytych zbiorów pełne parametry generatora razem z ziarnem oraz listę eksperymentów, które z niego korzystają. Każdy pomiar można więc odtworzyć, wywołując `sigscan-gen` z tymi parametrami, a powstałe pliki będą bajtowo identyczne na macOS i na Linuksie.

Pliki wyników. Każdy wiersz pliku CSV opisuje jedno powtórzenie. Zawiera nazwę implementacji i jej wariantu, układ automatu, tryb wyniku, nazwę zbioru danych, liczbę wątków lub procesów, rozmiar kawałka lub bloku, rozmiar tekstu, liczbę i długości wzorców, rozmiar automatu, liczbę bajtów zakładek, numer powtórzenia oraz czasy w milisekundach: odczytu pliku, budowy automatu, kopiowania na kartę, skanu, kopiowania z karty, scalania, komunikacji i całości. Na końcu są liczba trafień, suma kontrolna, nazwa maszyny, dodatkowe informacje o wariancie, takie jak czasy faz MPI czy wymiary siatki CUDA, oraz nazwa eksperymentu i opis punktu pomiarowego.

B Interfejs programów

Wszystkie programy obsługują opcję `--help`, która wypisuje pełny opis opcji z wartościami domyślnymi. Opcje generatora zawiera Tabela 1. Programy wyszukujące mają wspólne opcje z tabeli 6, a ich opcje własne zebraliśmy w tabeli 7. Wersję MPI uruchamia się poleceniem `mpirun -np P sigscan-mpi` z tymi samymi opcjami co pozostałe programy.

Tabela 6: Opcje wspólne programów `sigscan-seq`, `sigscan-omp`, `sigscan-mpi` i `sigscan-cuda`.

Opcja	Znaczenie
`--text`, `--patterns`	plik z tekstem i plik ze wzorcami
`--mode`	`count` zwraca liczbę trafień i sumę kontrolną, `list` pełną listę par
`--layout`	układ automatu na CPU: `dense` albo `compressed`
`--out`	plik CSV, do którego trafia lista trafień w trybie `list`
`--reps`, `--warmup`	liczba mierzonych powtórzeń i przebiegów rozgrzewkowych
`--csv`, `--csv-header`	wypisanie jednego wiersza CSV na powtórzenie i nagłówka
`--quiet`	brak czytelnego podsumowania na standardowym wyjściu błędów

Tabela 7: Opcje własne poszczególnych programów.

Program	Opcja	Znaczenie
`sigscan-seq`	`--algo`	`ac` to Aho-Corasick, `pfac` emulacja algorytmu GPU na CPU
	`--interleave`	liczba przeplatanych ścieżek automatu: 1, 2, 4, 8 lub 16
	`--stats`	rozmiary automatów i statystyki częściowych dopasowań
`sigscan-omp`	`--threads`	liczba wątków
	`--schedule`	`blocks`, `static`, `dynamic` albo `guided`
	`--chunk`	rozmiar kawałka dla harmonogramów innych niż `blocks`
	`--interleave`	jak w wersji sekwencyjnej
`sigscan-cuda`	`--kernel`	`global`, `ldg`, `smem-text`, `smem-root` albo `smem-all`
	`--block`, `--grid-mult`	liczba wątków w bloku i liczba bloków na multiprocesor
	`--streams`, `--chunk`	liczba strumieni i rozmiar kawałka w trybie potokowym
	`--warmup-ms`	czas pracy GPU przed pomiarem, domyślnie 2000 ms
	`--verify`	porównanie wyniku z referencją CPU

Bibliografia

[1] Martin Roesch. “Snort – Lightweight Intrusion Detection for Networks”. W: *Proceedings of the 13th USENIX Conference on System Administration (LISA ’99)*. 1999, s. 229–238. URL: https://www.usenix.org/legacy/event/lisa99/roesch.html.

[2] Open Information Security Foundation. *Suricata: Network Threat Detection Engine*. https://suricata.io.

[3] Alfred V. Aho i Margaret J. Corasick. “Efficient String Matching: An Aid to Bibliographic Search”. W: *Communications of the ACM* 18.6 (1975), s. 333–340. DOI: 10.1145/360825.360855.

[4] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. 2021. URL: https://www.openmp.org/specifications/.

[5] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. 2023. URL: https://www.mpi-forum.org/docs/.

[6] NVIDIA. *CUDA Programming Guide*. 2025. URL: https://docs.nvidia.com/cuda/cuda-programming-guide/.

[7] Cheng-Hung Lin, Chen-Hsiung Liu, Lung-Sheng Chien i Shih-Chieh Chang. “Accelerating Pattern Matching Using a Novel Parallel Algorithm on GPUs”. W: *IEEE Transactions on Computers* 62.10 (2013), s. 1906–1916. DOI: 10.1109/TC.2012.254.

[8] Antonino Tumeo, Oreste Villa i Donatella Sciuto. “Efficient Pattern Matching on GPUs for Intrusion Detection Systems”. W: *Proceedings of the 7th ACM International Conference on Computing Frontiers*. 2010, s. 87–88. DOI: 10.1145/1787275.1787296.

[9] Daniel Lemire. “Fast Random Integer Generation in an Interval”. W: *ACM Transactions on Modeling and Computer Simulation* 29.1 (2019), 3:1–3:12. DOI: 10.1145/3230636.

[10] Kitware. *CMake, wersja 4.4 (Mac) i 3.31 (Colab)*. https://cmake.org.

[11] Viktor Kirilov. *doctest: The fastest feature-rich C++ single-header testing framework, wersja 2.4.11*. https://github.com/doctest/doctest.

[12] NumPy Developers. *NumPy, wersja 2.5*. https://numpy.org.

[13] The pandas development team. *pandas, wersja 3.1*. https://pandas.pydata.org.

[14] The Matplotlib development team. *Matplotlib, wersja 3.11*. https://matplotlib.org.

[15] LLVM Project. *LLVM/OpenMP Runtime Library (libomp), wersja 20.1*. https://openmp.llvm.org.

[16] NVIDIA. *CUDA Toolkit, wersja 13.0*. https://developer.nvidia.com/cuda-toolkit.

[17] The Open MPI Project. *Open MPI: Open Source High Performance Computing, wersja 5.0*. https://www.open-mpi.org.

[18] Google. *Colab CLI: command-line interface for Google Colab*. `https://github.com/googlecolab/google-colab-cli`. 2026.